

# C++ 개요 및 실습 환경 설정

**김미수**

전남대학교 인공지능학부

misoo.kim@jnu.ac.kr

<https://sites.google.com/view/semi-jnu>

# C++ / CPP 이란?

# C++를 왜 배워야 하는가? (1/3)

- 점유율 Top-10 언어 (출처: TIOBE, 2026.09)

| Aug 2026 | Aug 2025 | Change | Programming Language                                                                  |                  | Ratings | Change |
|----------|----------|--------|---------------------------------------------------------------------------------------|------------------|---------|--------|
| 1        | 1        |        |    | Python           | 18.53%  | -7.61% |
| 2        | 3        | ▲      |    | C                | 11.10%  | +2.07% |
| 3        | 2        | ▼      |    | C++              | 8.62%   | -0.56% |
| 4        | 4        |        |    | Java             | 8.25%   | -0.34% |
| 5        | 5        |        |    | C#               | 4.09%   | -1.43% |
| 6        | 6        |        |    | JavaScript       | 2.63%   | -0.52% |
| 7        | 7        |        |   | Visual Basic     | 2.18%   | -0.15% |
| 8        | 12       | ▲▲     |  | SQL              | 1.88%   | +0.16% |
| 9        | 14       | ▲▲     |  | R                | 1.56%   | +0.19% |
| 10       | 18       | ▲▲     |  | Rust (C/C++ 대체제) | 1.45%   | +0.31% |

# C++를 왜 배워야 하는가? (2/3)

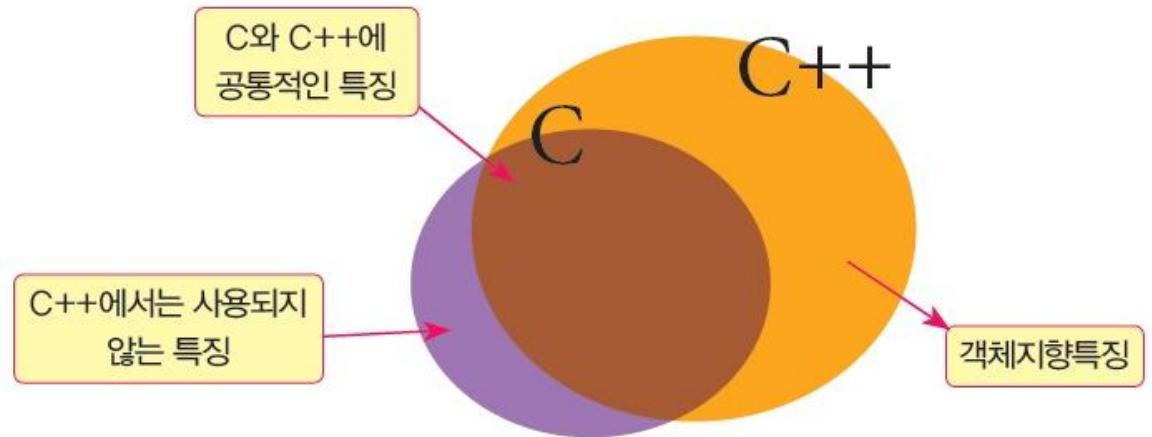
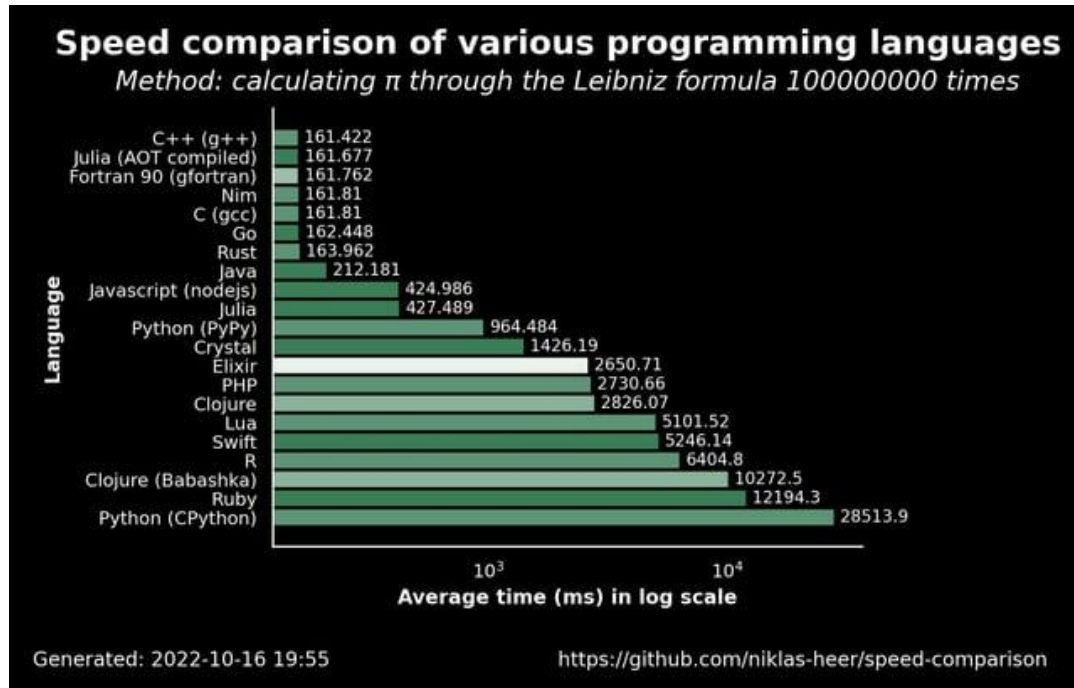
- Top Programming Language (IEEE Spectrum, 2020)
  - C++은 어디에 사용되는가?

Language Ranking: **Trending**

| Rank | Language   | Type                                                                                                                                                                                                                                                              | Score |
|------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 1    | Python     |          | 100.0 |
| 2    | Java       |          | 94.9  |
| 3    | C          |          | 91.4  |
| 4    | C++        |    | 87.5  |
| 5    | JavaScript |                                                                                                                                                                              | 74.9  |

# C++를 왜 배워야 하는가? (3/3)

- CPP = C + 객체지향 개념
  - C의 장점을 계승 : 빠른 속도, 하드웨어 접근, 긴 역사의 도구들 ...
  - 객체 지향: 코드 재사용성과 프로그램 유지보수 측면에 강점

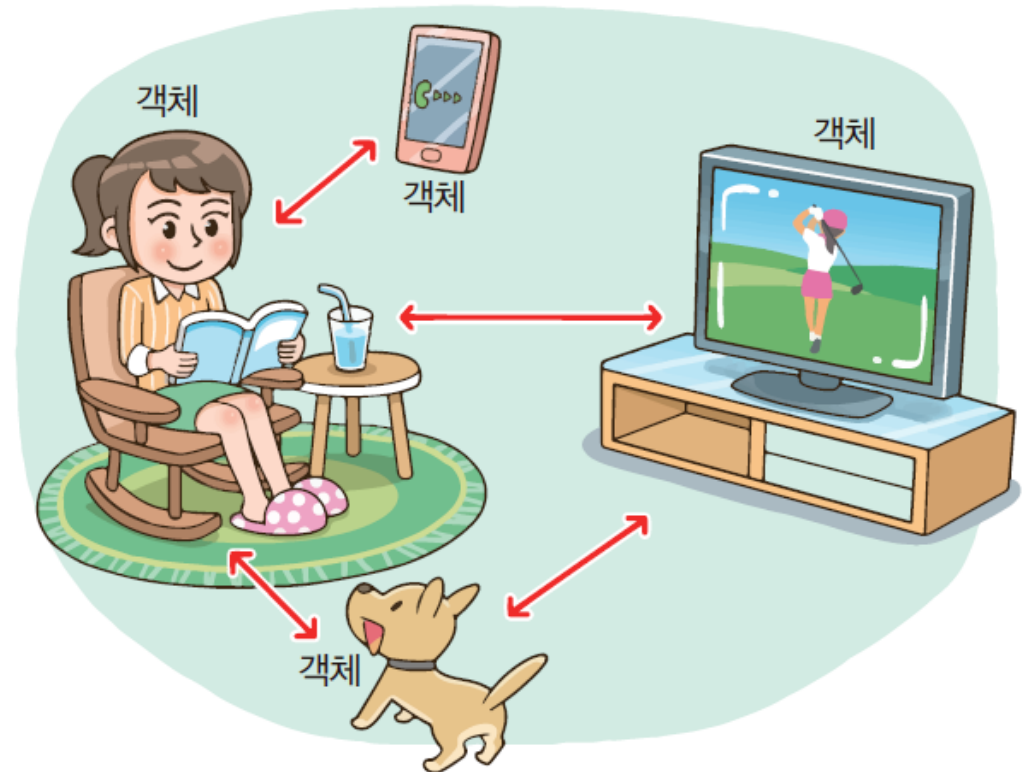
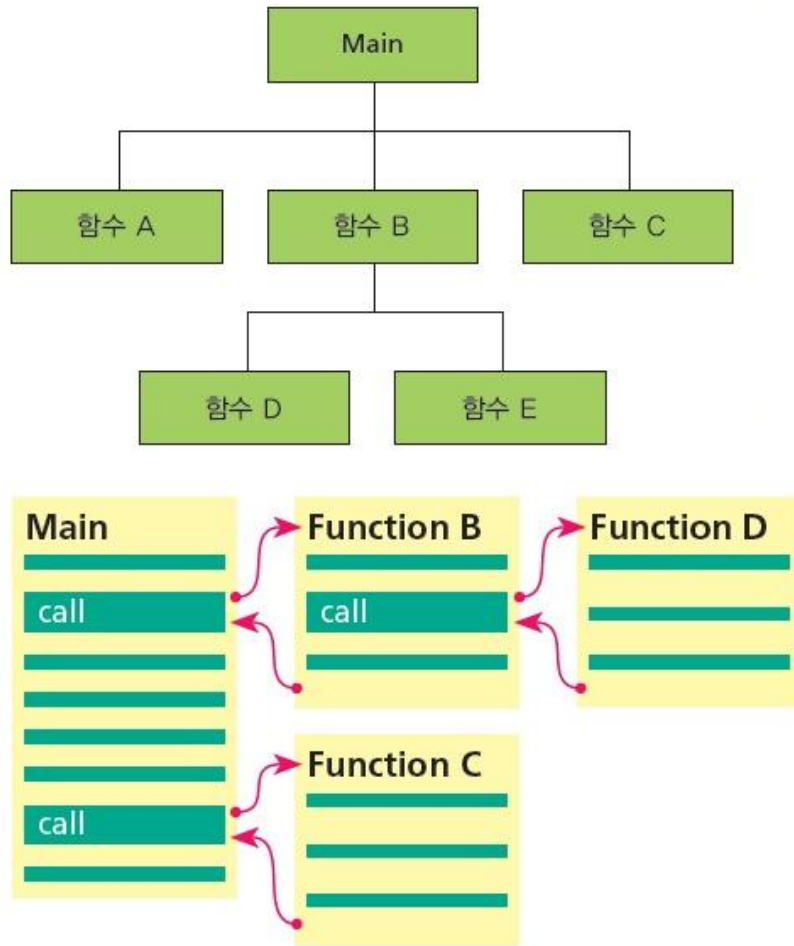


# C++ 특징 (1/2)

- 클래스(class)
  - 클래스를 이용하여 하나의 객체의 속성과 동작들을 한곳으로 모아서 정의
- 상속(inheritance)
  - 클래스를 상속받아서 기존의 코드를 재사용
- 연산자 중복(operator overloading)
  - 대상에 따라서 동일한 연산자로 새로운 연산을 정의
- 함수 중복(function overloading)
  - 매개 변수만 다르면 동일한 이름의 함수를 여러 개 생성
- new와 delete 연산자
  - 동적 메모리 할당과 해제를 담당하는 연산자
- 제네릭(generics)
  - 클래스 정의를 자료형에 상관없이 재사용

# C++ 특징 (2/2)

- 멀티 패러다임 프로그래밍 지원: 절차지향 + 객체지향



# 실습 환경 설정

# 실습 환경: Window + VS Code

- 절차

1. MinGW (c/c++ 컴파일러) 설치 후 설정
2. VS Code 설치 후 설정

# C/C++ 코딩 환경 구축

## 1. MinGW (Minimalist GNU for Windows) 설치

- MinGW: 리눅스/유닉스 계열에서 사용하던 g, g++, gdb 등의 툴체인을 윈도우 환경에서 사용하도록 하는 프로젝트
  - g/g++ : C/C++ 컴파일러, ld: 링커, gdb: 디버거
- <https://www.msys2.org/>
  - x86\_64 : window 버전

### Installation

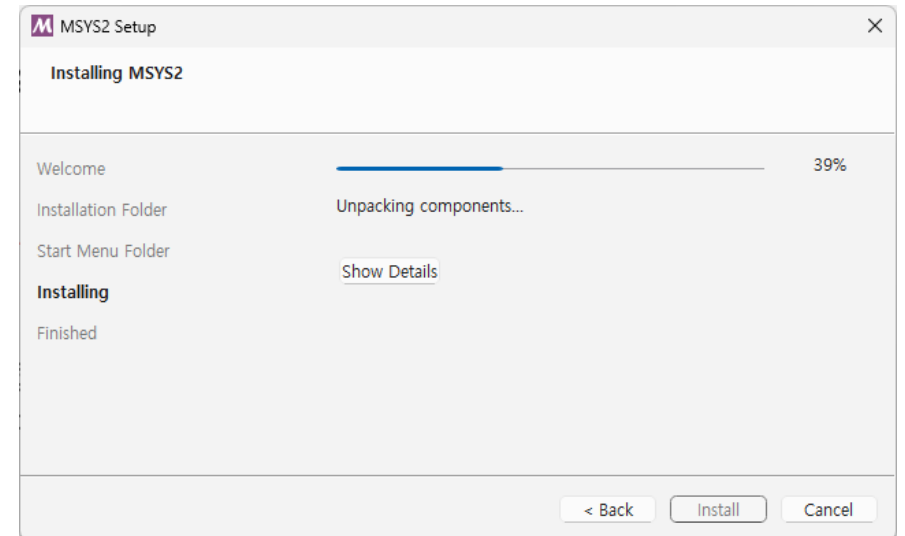
1. Download the installer:

[msys2-x86\\_64-20260611.exe](#) or [msys2-arm64-20260611.exe](#) (ARM64)

(Optional) For more information on the installer, like command line options, or how to verify the checksum and signature of the installer, see the [installer guide](#).

2. Run the installer. Installing MSYS2 requires 64 bit Windows 10 (1809+) or newer.

3. Enter your desired **Installation Folder**. We recommend using the default, see [installation folder recommendations](#) for more information.



# C/C++ 코딩 환경 구축

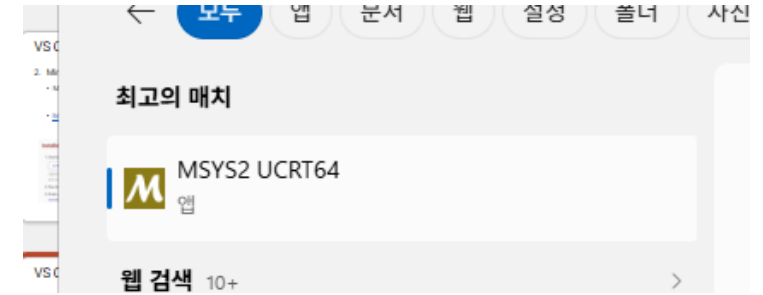
## 2. MinGW toolchain 설치 후 확인

- (run 체크를 안해서, 실행이 안된다면 MSYS2 UCRT64 실행 후 입력)
- `pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain`
- `gcc --version`, `g++ --version`, `gdb --version`

```
user@ROSE-JNU-PC5090 UCRT64 ~
$ pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
:: There are 13 members in group mingw-w64-ucrt-x86_64-toolchain:
:: Repository ucrt64
 1) mingw-w64-ucrt-x86_64-binutils 2) mingw-w64-ucrt-x86_64-crt 3) mingw-w64-ucrt-x86_64-gcc
 4) mingw-w64-ucrt-x86_64-gdb 5) mingw-w64-ucrt-x86_64-gdb-multiarch
 6) mingw-w64-ucrt-x86_64-headers 7) mingw-w64-ucrt-x86_64-libmangle
 8) mingw-w64-ucrt-x86_64-libwinpthread 9) mingw-w64-ucrt-x86_64-make
10) mingw-w64-ucrt-x86_64-pkgconf 11) mingw-w64-ucrt-x86_64-tools
12) mingw-w64-ucrt-x86_64-winthreads 13) mingw-w64-ucrt-x86_64-winstorecompat

Enter a selection (default=all):
resolving dependencies...
looking for conflicting packages...

Packages (55) binutils-2.46-2 bison-3.8.2-5 diffstat-1.69-1 diffutils-3.12-1 dos2unix-7.5.6-1
flex-2.6.4-4 m4-1.4.21-1 make-4.4.1-3 mingw-w64-ucrt-x86_64-bzip2-1.0.8-3
mingw-w64-ucrt-x86_64-expat-2.8.1-2 mingw-w64-ucrt-x86_64-gcc-libs-16.1.0-5
mingw-w64-ucrt-x86_64-gettext-runtime-1.0-1 mingw-w64-ucrt-x86_64-gmp-6.3.0-2
mingw-w64-ucrt-x86_64-isl-0.27-1 mingw-w64-ucrt-x86_64-libb2-0.98.1-3
mingw-w64-ucrt-x86_64-libffi-3.5.2-1 mingw-w64-ucrt-x86_64-libiconv-1.19-1
mingw-w64-ucrt-x86_64-libsystre-1.0.2-2 mingw-w64-ucrt-x86_64-libtre-0.9.0-2
mingw-w64-ucrt-x86_64-mpc-1.4.1-1 mingw-w64-ucrt-x86_64-mpdecimal-4.0.1-3
mingw-w64-ucrt-x86_64-mpfr-4.2.2-3 mingw-w64-ucrt-x86_64-ncurses-6.6-4
mingw-w64-ucrt-x86_64-openssl-3.6.3-1 mingw-w64-ucrt-x86_64-pcre2-10.47-1
mingw-w64-ucrt-x86_64-python-3.14.5-1 mingw-w64-ucrt-x86_64-readline-8.3.003-1
```



```
user@ROSE-JNU-PC5090 UCRT64 ~
$ gcc --version
gcc.exe (Rev5, Built by MSYS2 project) 16.1.0
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

user@ROSE-JNU-PC5090 UCRT64 ~
$ g++ --version
g++.exe (Rev5, Built by MSYS2 project) 16.1.0
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

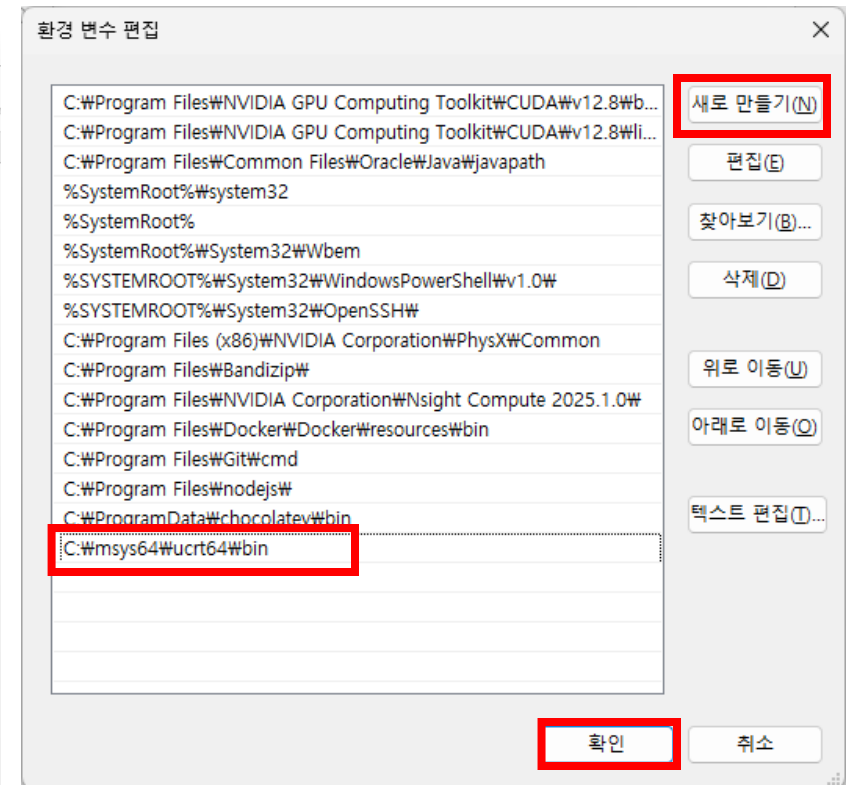
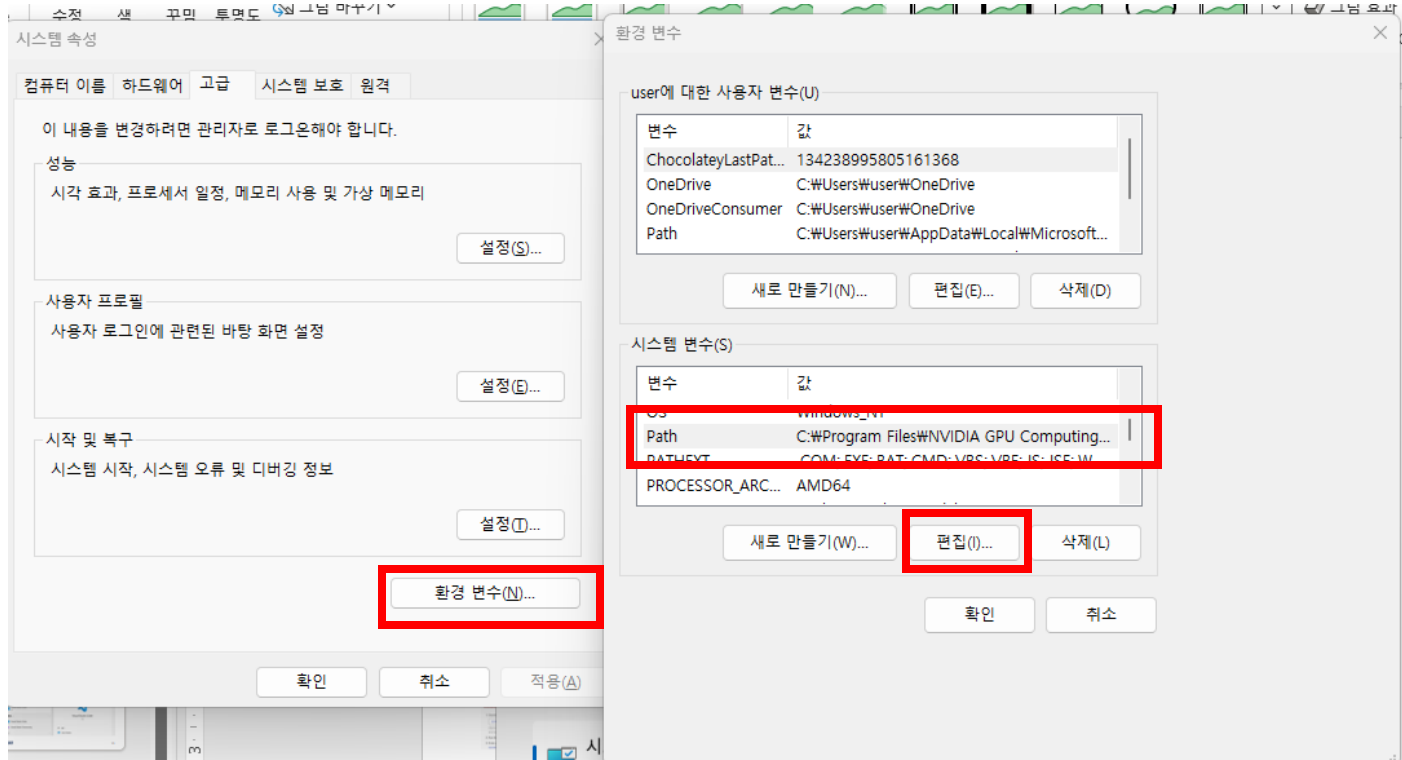
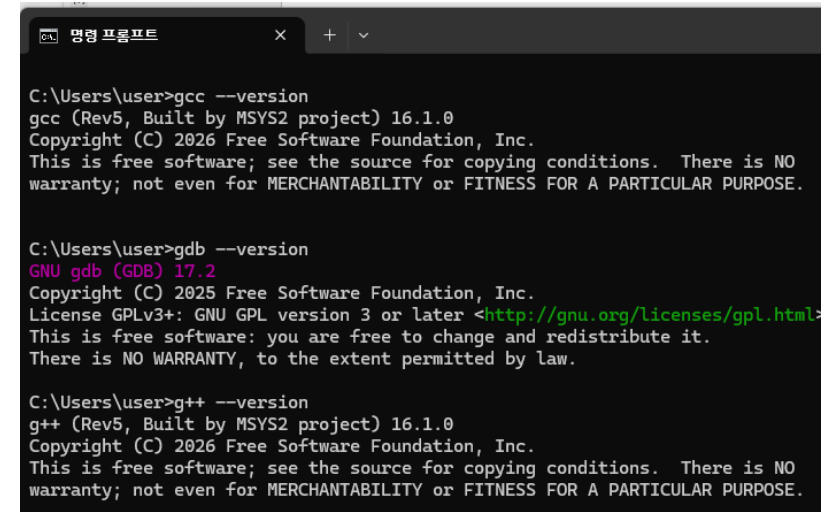
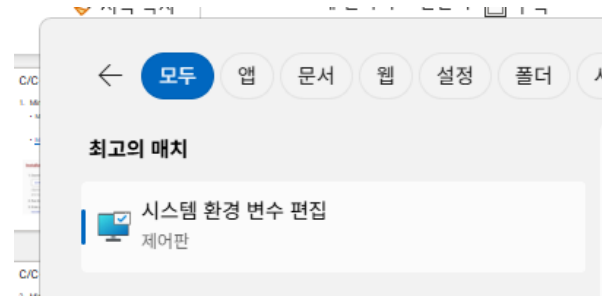
user@ROSE-JNU-PC5090 UCRT64 ~
$ gdb --version
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

user@ROSE-JNU-PC5090 UCRT64 ~
$
```

# C/C++ 코딩 환경 구축

## 3. MinGW 환경 변수 설정 후 확인

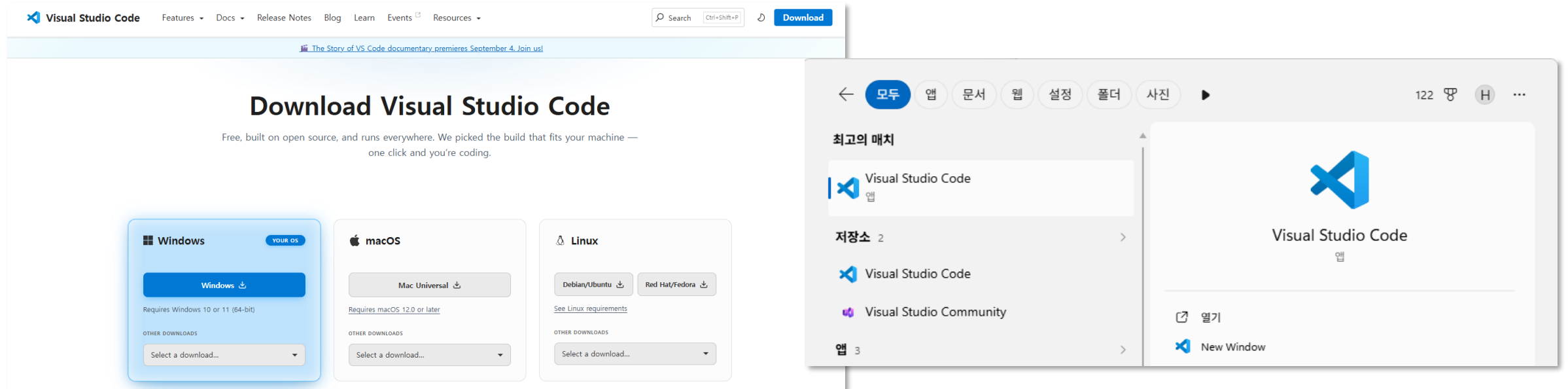
- (환경 변수 설정: 시스템 어디서든 해당 위치에 접근하기 위한 것)
- C:\msys64\ucrt64\bin 를 PATH에 추가



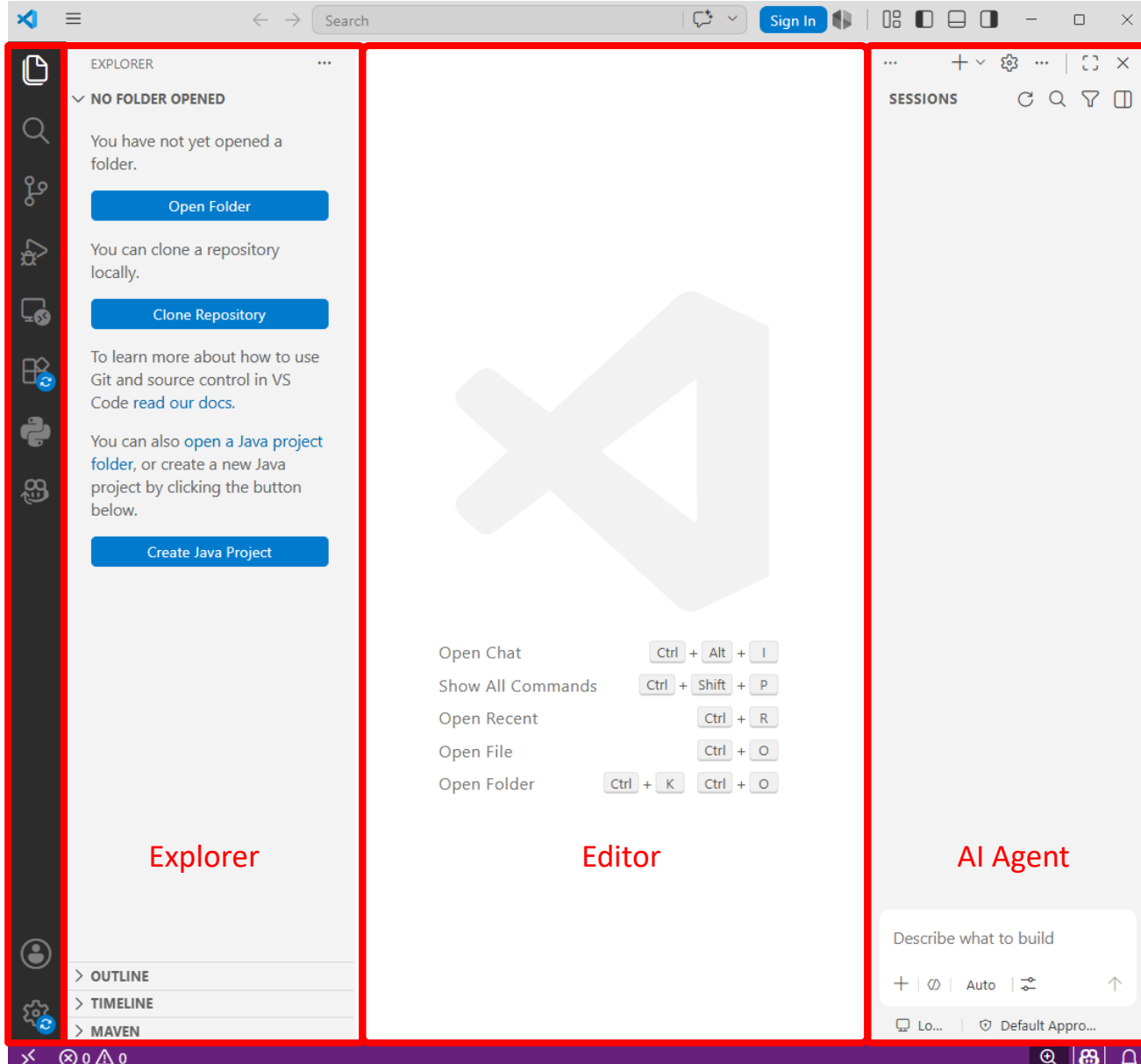
# C/C++ 코딩 환경 구축

## 4. Visual Studio Code (VS Code) 설치

- <https://code.visualstudio.com/>
  - 과거에는 C/C++ 개발을 위해 Visual Studio를 사용함 → 이를 간소화한 “텍스트 에디터”
  - VS Code에 여러 플러그인과 컴파일러를 “연결해서” 프로그래밍과 프로젝트를 관리함

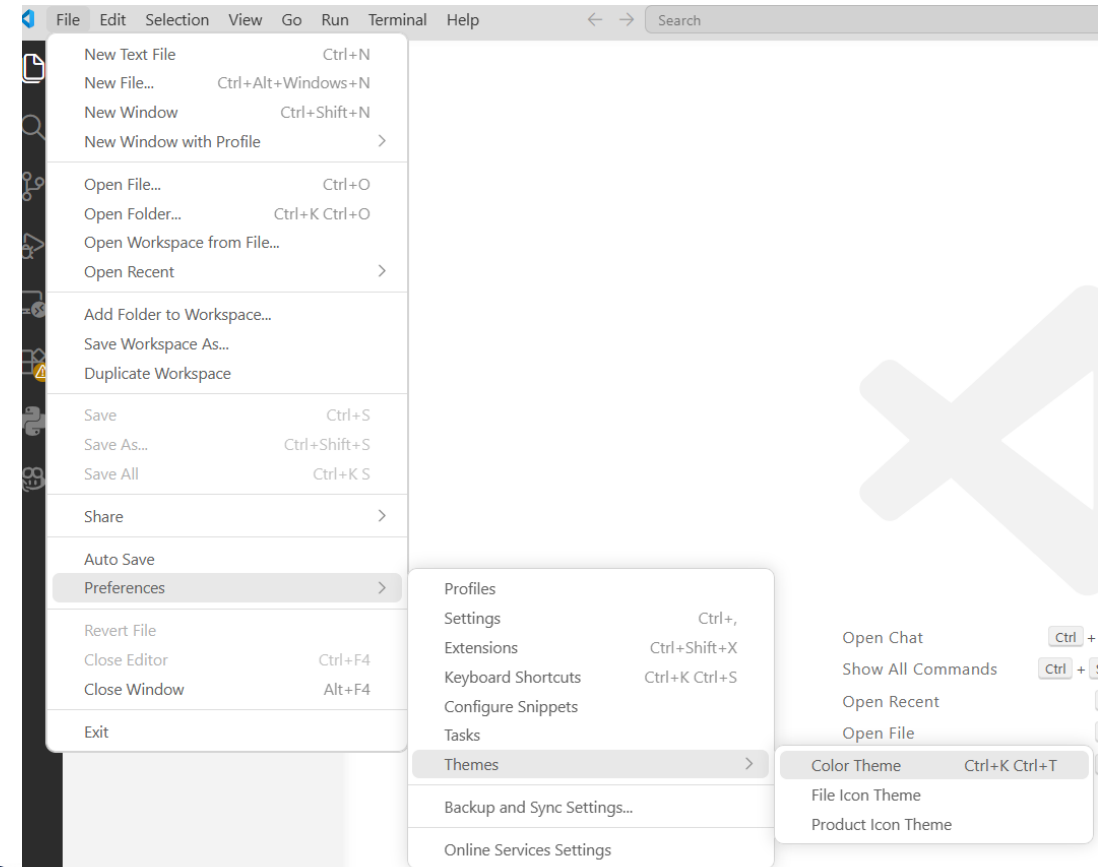


# C/C++ 코딩 환경 구축



## • 테마 변경

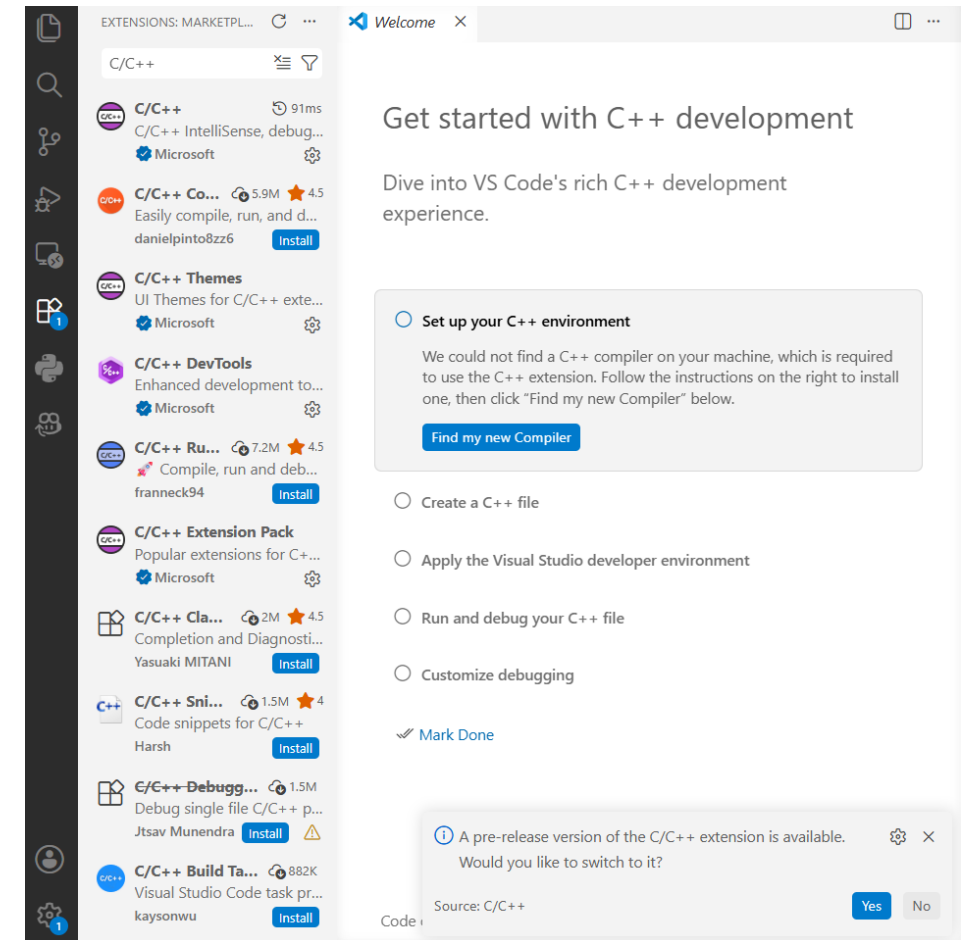
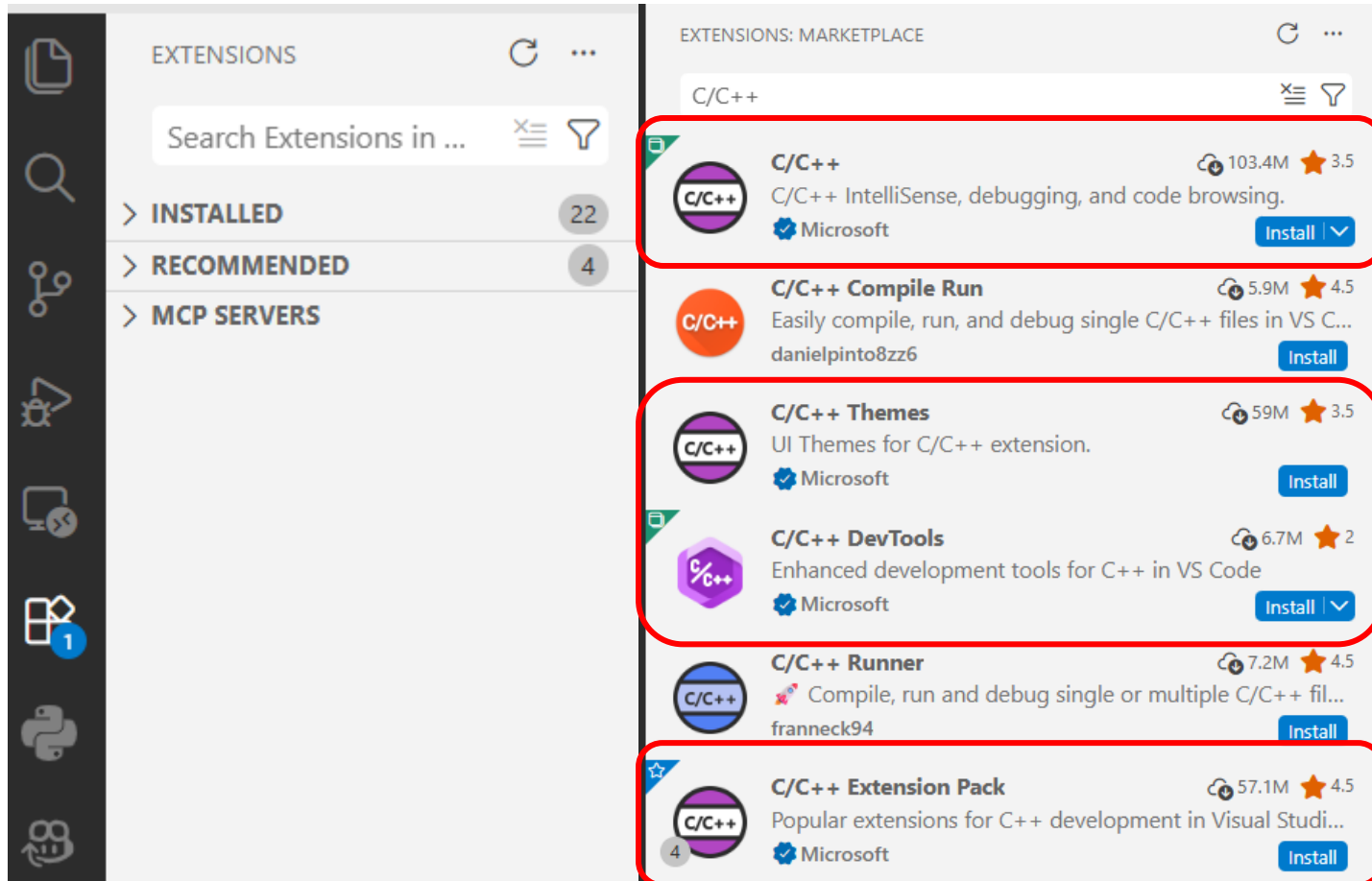
- File → Preferences → Themes



# C/C++ 코딩 환경 구축

## 5. 마켓플레이스에서 C/C++ 확장 프로그램 설치

- Extensions → C/C++ 검색 → Microsoft에서 만든 C/C++ 관련 플러그인 설치



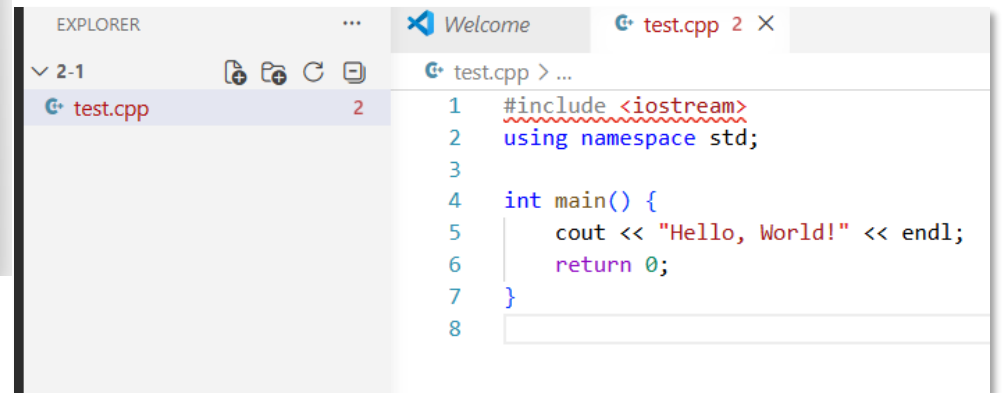
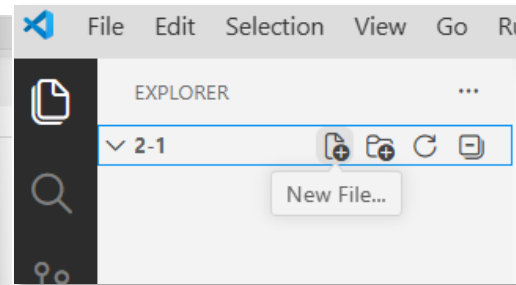
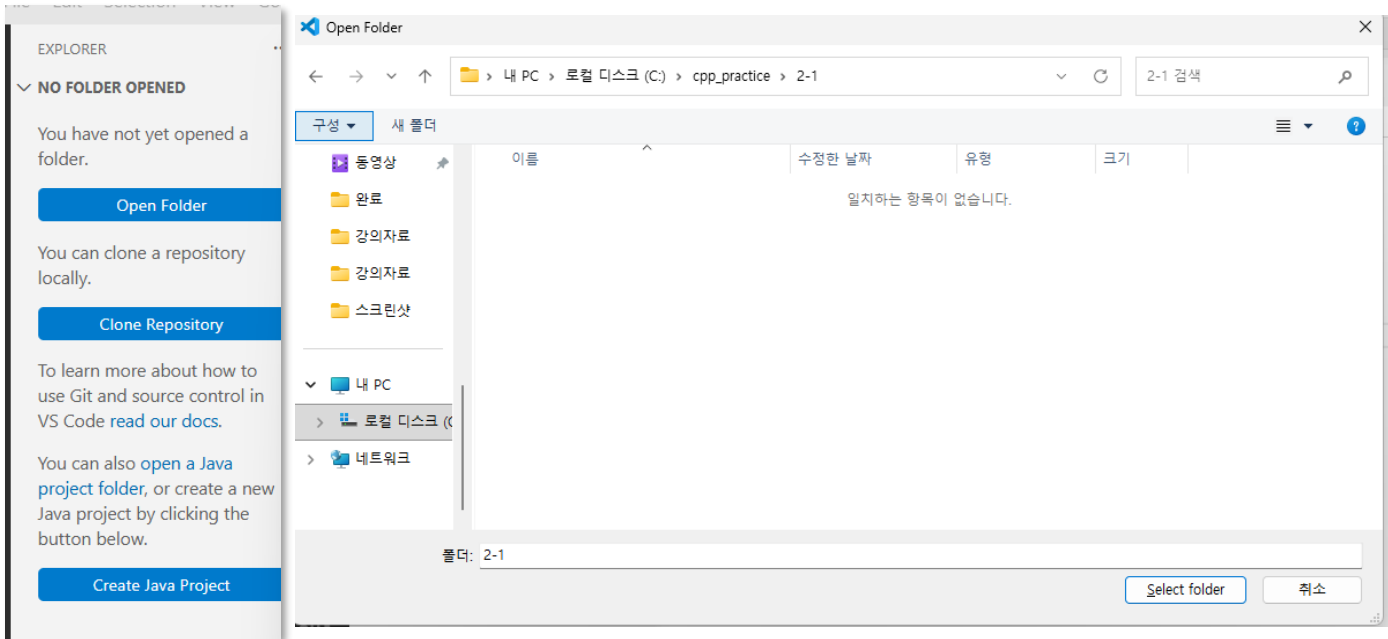
# C/C++ 코딩 환경 구축

## 6. 간단한 프로그램 작성하기

- 작업 폴더 지정 → 파일 생성 → 코드 입력 후 저장
- (되도록이면 영문 폴더에)

```
#include <iostream>
using namespace std;

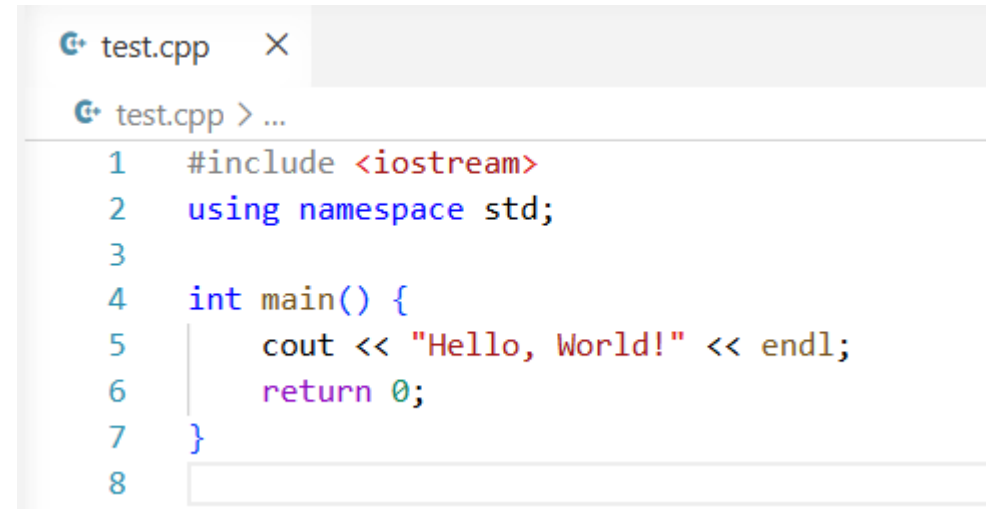
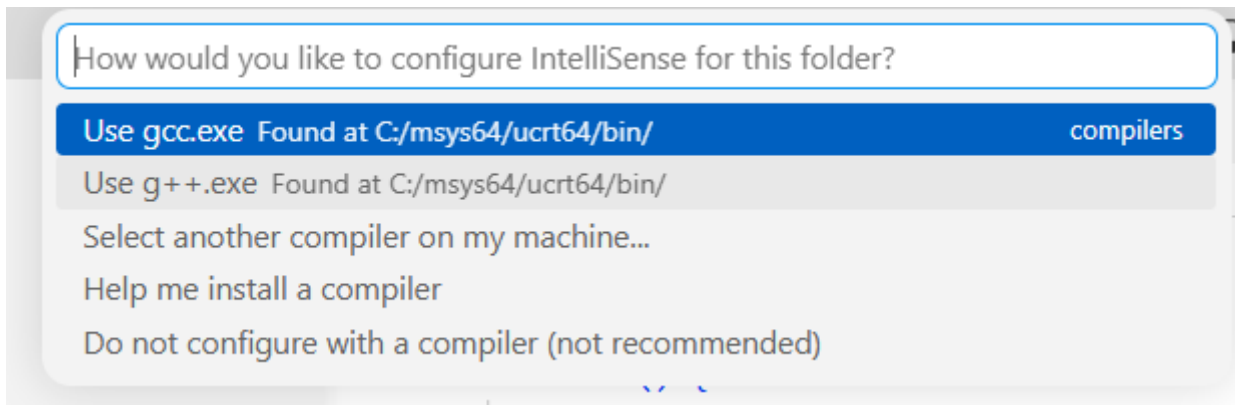
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```



# C/C++ 코딩 환경 구축

## 7. 컴파일러 연결해주기

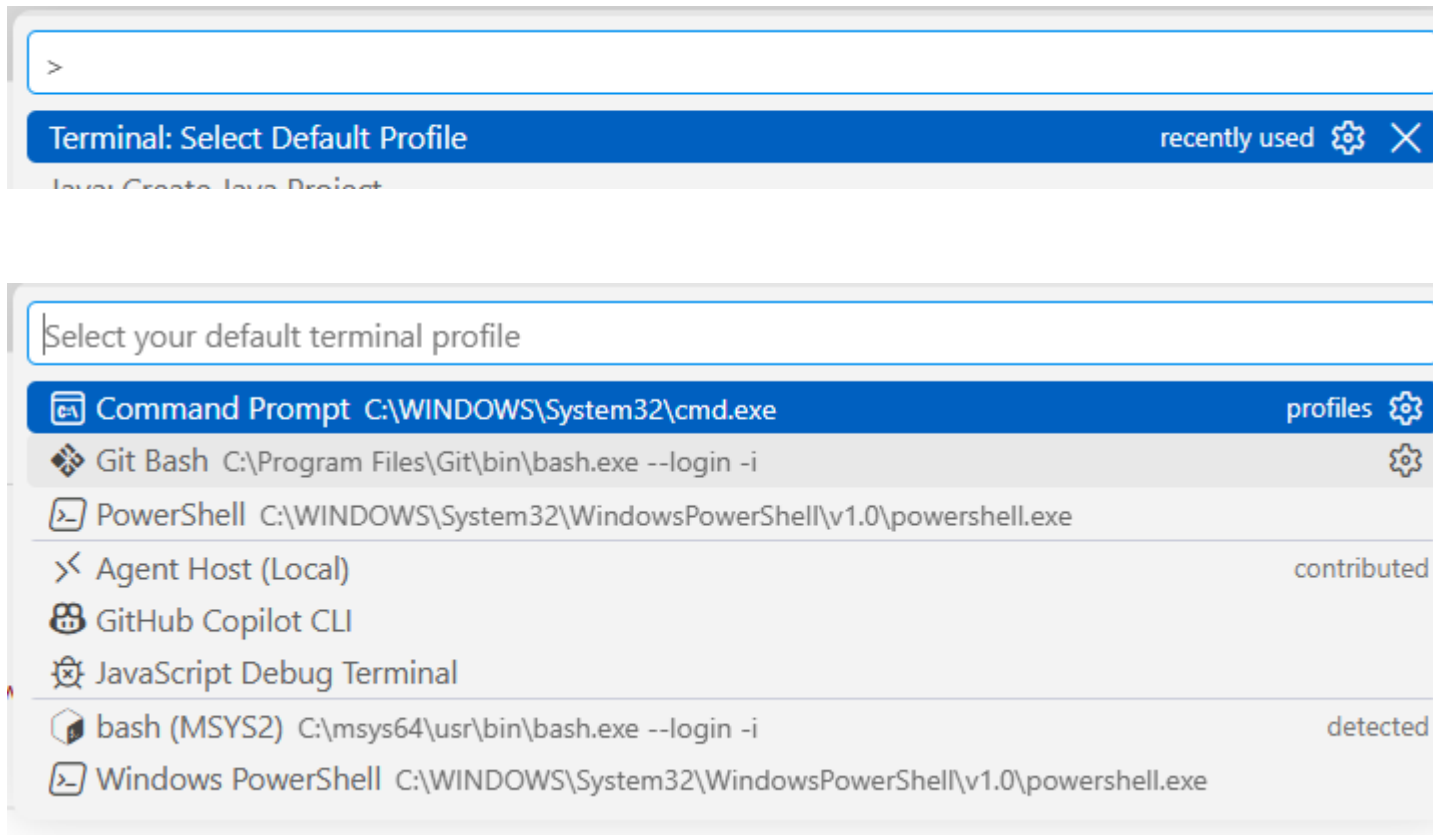
- 하단에 configure ~ → g++ 선택



# C/C++ 코딩 환경 구축

## 8. terminal을 윈도우 기본 커맨드창으로 바꾸기 (이건 취향)

- ctrl + shift + P → command Prompt

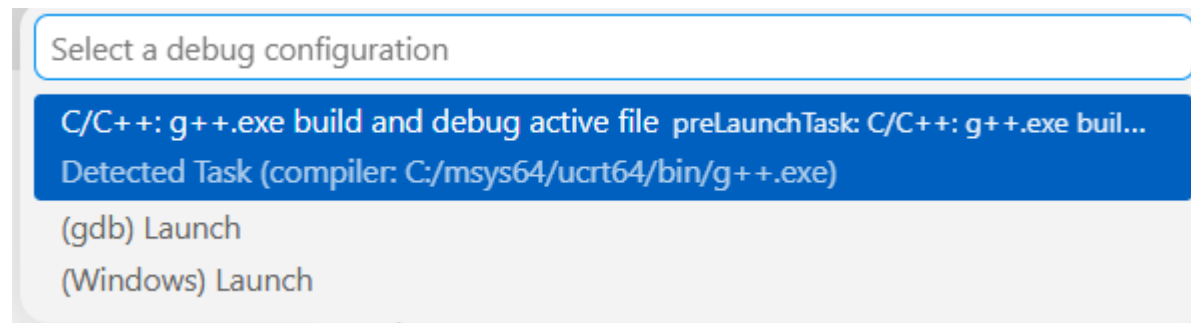


# C/C++ 코딩 환경 구축

## 9. 실행해보기



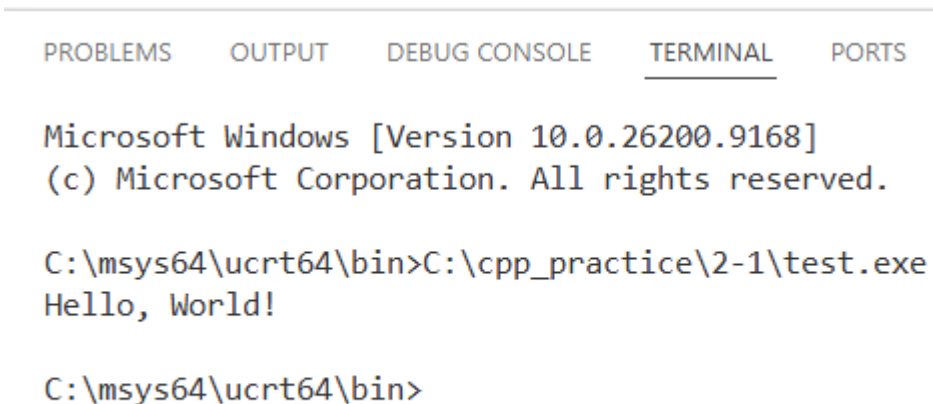
```
test.cpp x
test.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      cout << "Hello, World!" << endl;
6      return 0;
7  }
8
```



Select a debug configuration

C/C++: g++.exe build and debug active file preLaunchTask: C/C++: g++.exe buil...  
Detected Task (compiler: C:/msys64/ucrt64/bin/g++.exe)

(gdb) Launch  
(Windows) Launch

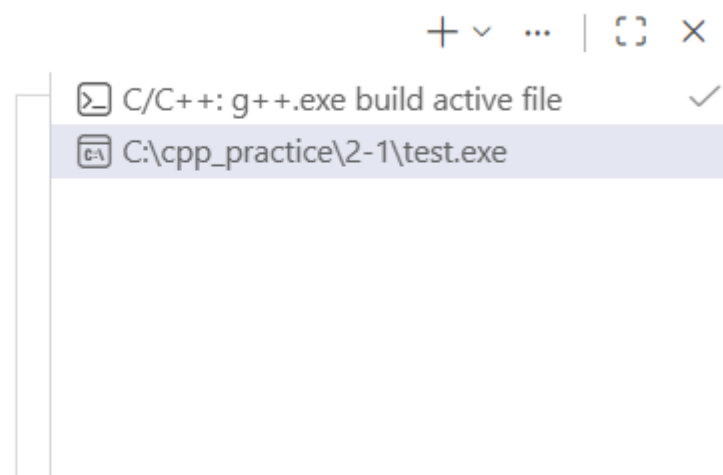


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Microsoft Windows [Version 10.0.26200.9168]  
(c) Microsoft Corporation. All rights reserved.

C:\msys64\ucrt64\bin>C:\cpp\_practice\2-1\test.exe  
Hello, World!

C:\msys64\ucrt64\bin>



+ v ... | [ ] x

- ☑ C/C++: g++.exe build active file ✓
- 📄 C:\cpp\_practice\2-1\test.exe

# C/C++ 코딩 환경 구축

- 어떤 일이 일어났는가?

EXPLORER

- 2-1
  - .vscode
  - test.cpp
  - test.exe

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

\* Executing task: C/C++: g++.exe build active file

Starting build...

```
cmd /c chcp 65001>nul && C:/msys64/ucrt64/bin/g++.exe -fdiagnostics-color=always -g C:\cpp_practice\2-1\test.cpp -o C:\cpp_practice\2-1\test.exe
```

Build finished successfully.

\* Terminal will be reused by tasks, press any key to close it.

C/C++: g++.exe build active file ✓

C:\cpp\_practice\2-1\test.exe

Microsoft Windows [Version 10.0.26200.9168]  
(c) Microsoft Corporation. All rights reserved.

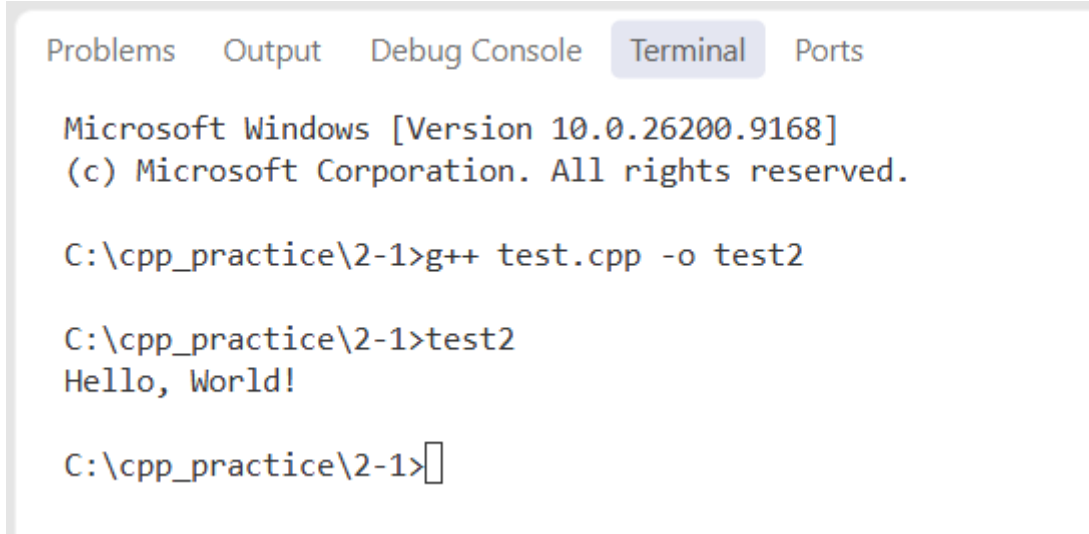
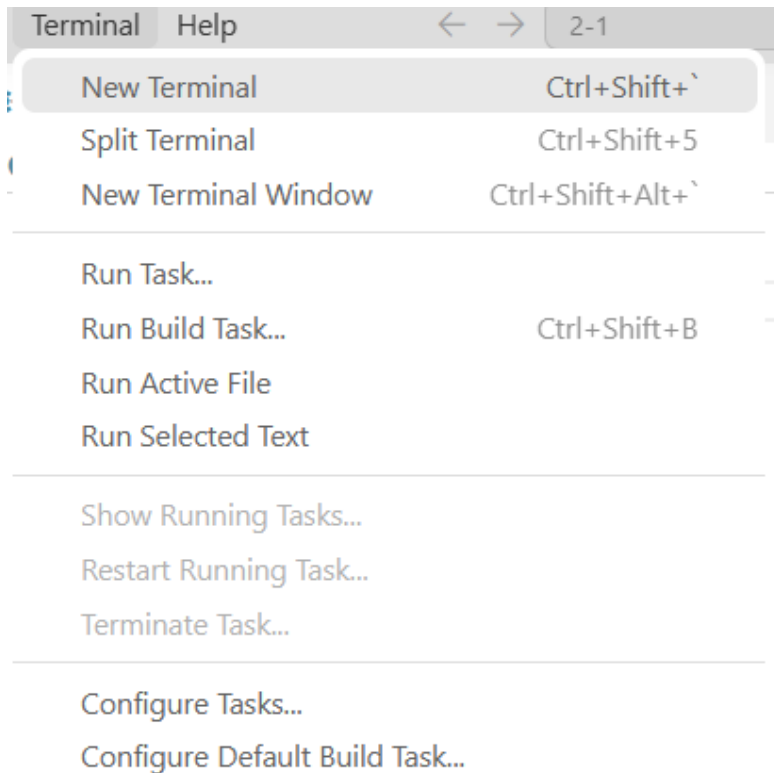
```
C:\msys64\ucrt64\bin>C:\cpp_practice\2-1\test.exe
```

Hello, World!

```
C:\msys64\ucrt64\bin>
```

# C/C++ 코딩 환경 구축

- 이렇게 한 거랑 같음
  - `g++ test.cpp -o test2`
  - `./test2.exe`



# C/C++은 어떻게 컴파일 되는가?

- [전처리 → 컴파일 (→ 어셈블) → 링크]

## 1. 전처리 (preprocessing)

#include, #define 등의 전처리 명령처리

## 2. 컴파일 (compile)

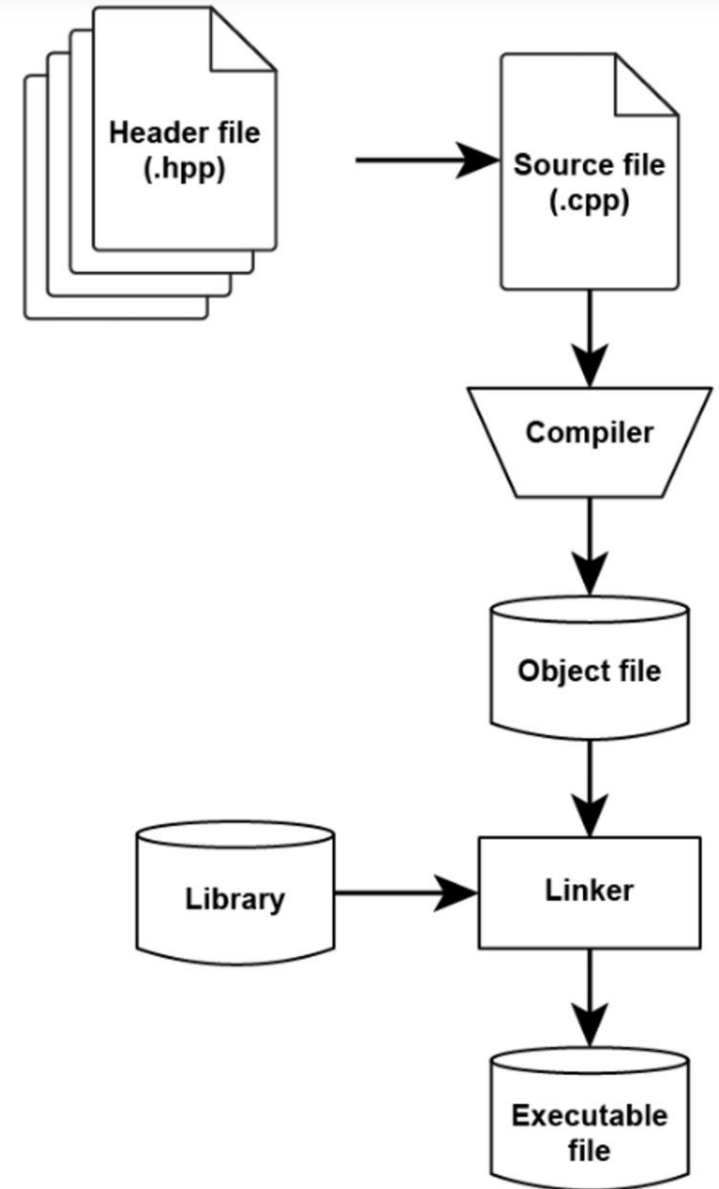
소스 코드를 어셈블리 코드로 변환

## 3. 어셈블 (assemble) (컴파일과 묶어서 보기도 함)

어셈블리 코드를 기계어(object file)로 변환

## 4. 링크 (link)

라이브러리와 Object File을 결합하여 실행 파일(.exe) 생성



# Git/GitHub 실습

# Git과 GitHub란?

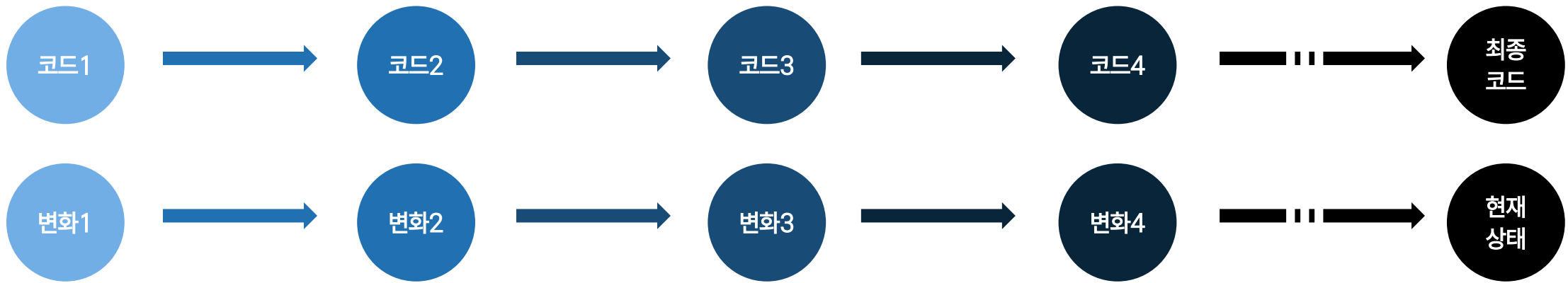
- Git: 버전 관리 시스템 (Version Control System: VCS)
  - 빠른 속도 & 저장소 지원
  - 오픈 소스 소프트웨어: 소스코드나 설계도가 공개된 소프트웨어
- GitHub: 대표적인 무료 Git 저장소
  - 버전관리, 백업, 협업을 지원



# Git과 GitHub란?

- 버전 관리

- 버전들을 미리 저장해, 이를 통해 이전 버전으로 되돌아갈 수 있도록 도와주는 시스템



모두 저장해 놓아야 함!



# Git과 GitHub란?

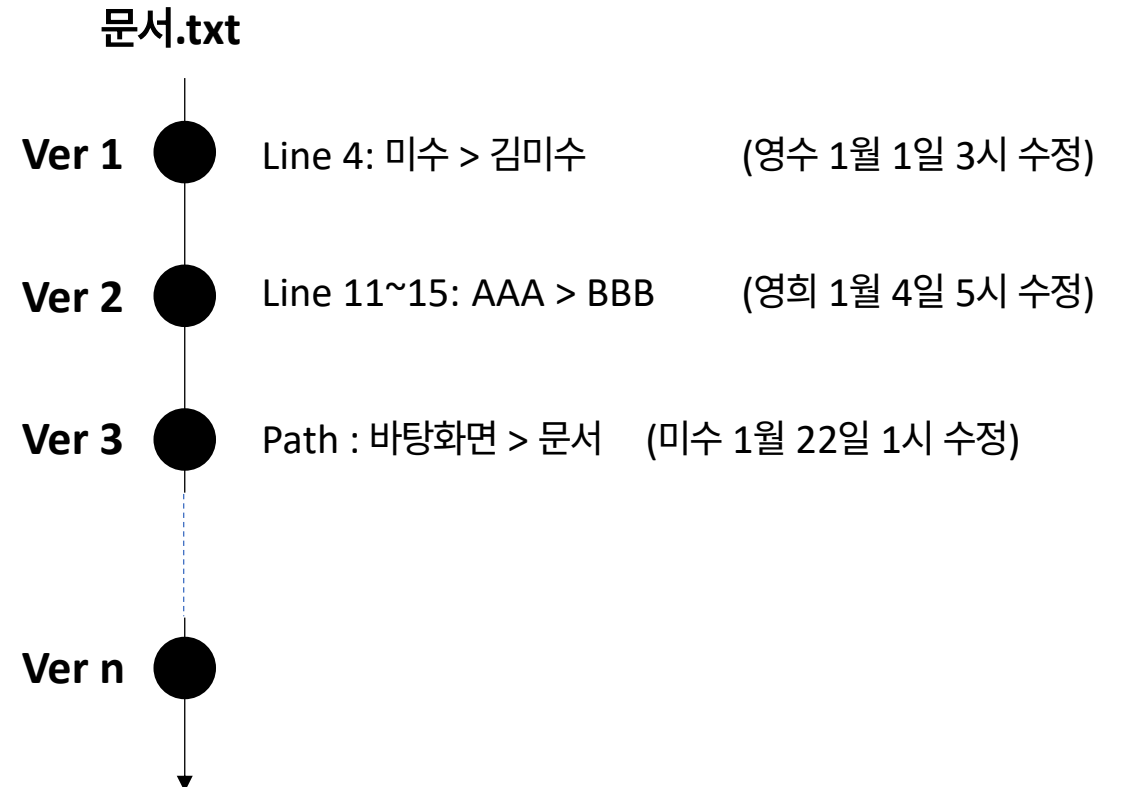
- 버전 관리

- 문서 수정 마다, 언제, 어떤 것을, 누가 변경했는지를 구체적으로 기록

| Name                |
|---------------------|
| 120525_문서_업데이트.txt  |
| 120604_문서.txt       |
| 120605_문서_수정판.txt   |
| 120605_문서_수정판2.txt  |
| 120605_문서_최신 복사.txt |
| 120605_문서_최신.txt    |
| 120605_문서.txt       |
| 1200602_문서.txt      |
| 문서_회의용.txt          |



[https://backlog.com/git-tutorial/kr/intro/intro1\\_1.html](https://backlog.com/git-tutorial/kr/intro/intro1_1.html)



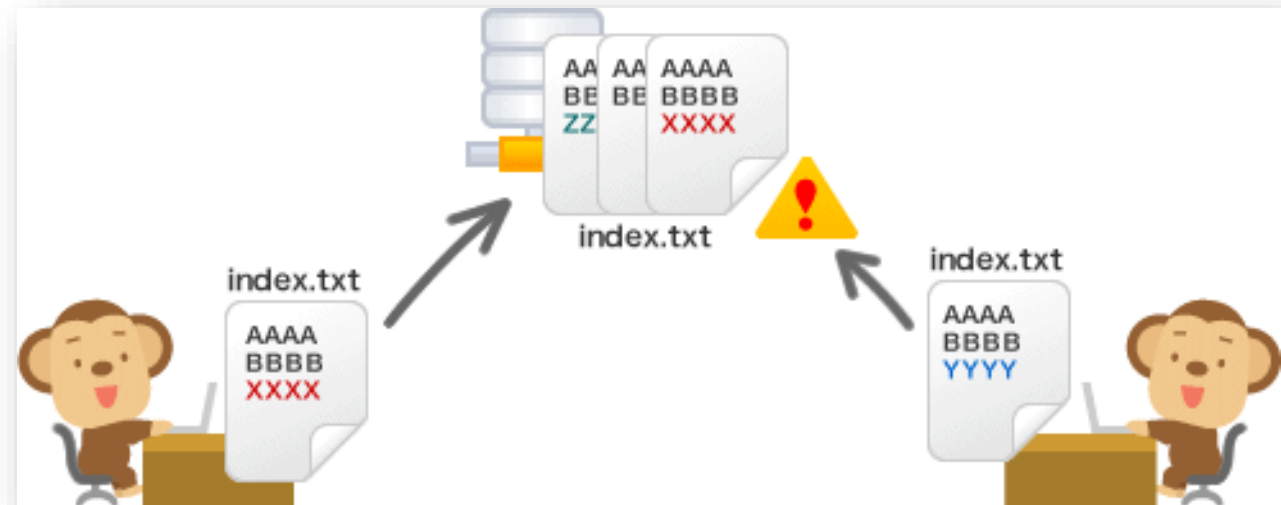
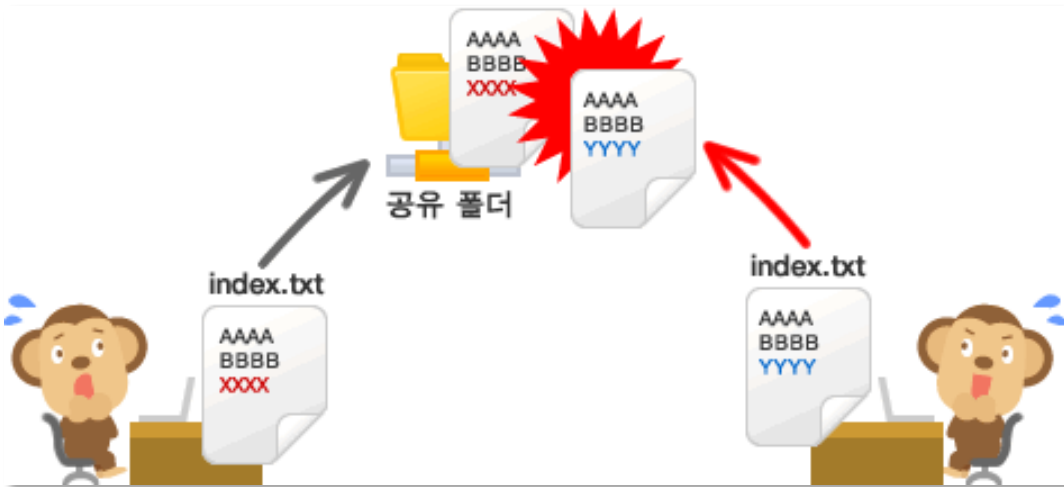
# Git과 GitHub란?

- 백업
  - 현재 컴퓨터에 있는 자료를 다른 컴퓨터에 복제
  - **GitHub**: 백업 공간을 제공하는 원격/온라인 저장소



# Git과 GitHub란?

- 협업
  - 온라인 서비스로 여러 개발자가 동시 접근 가능
  - 기록을 바탕으로, 오류 생겼을 때 파악 쉬움
  - **GitHub**: 협업 과정에서 발생하는 문제 처리 기능 제공



# 용어 정리 (1/5)

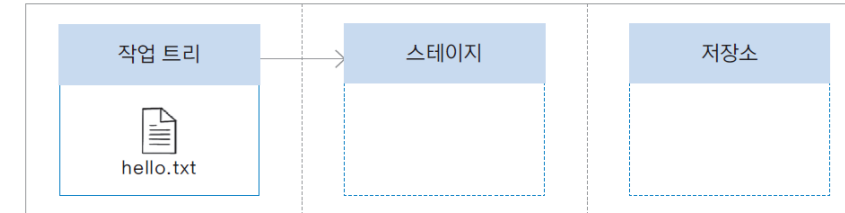
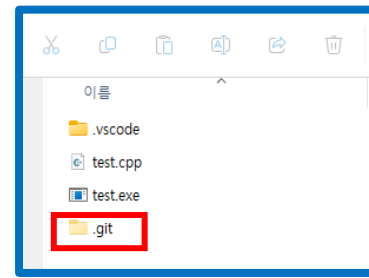
## • 구조

- **작업 트리**: 파일 수정, 저장 등의 작업을 하는 폴더
- **스테이지**: 버전으로 만들 파일이 대기하는 곳
- **저장소**: 버전 저장소

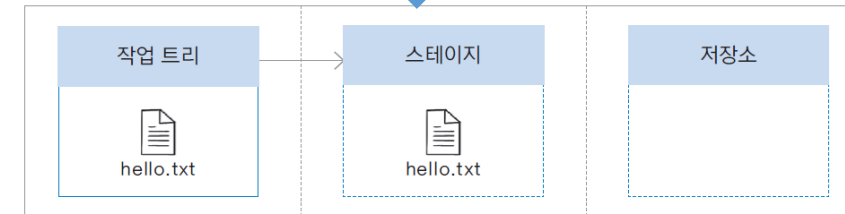
## • 행위

- **스테이징**: 버전으로 만들 파일을 스테이지로 옮기는 작업
- **커밋**: 스테이지에 있는 파일을 저장소로 보내는 작업  
= 특정 시점의 상태를 기록한 것 (스냅샷, 커밋이 모여서 릴리즈 버전이 됨)
- **푸시**: 지역 저장소에 있는 파일과 이력을 원격 저장소에 보내는 작업

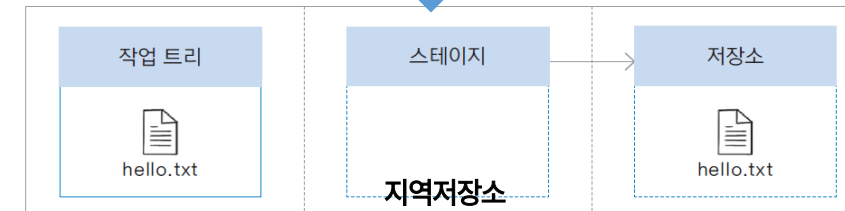
⇔ **풀**: 원격 저장소에 있는 파일과 이력을 지역 저장소로 가져옴



↓ **스테이징 (add)**



↓ **커밋 (commit)**

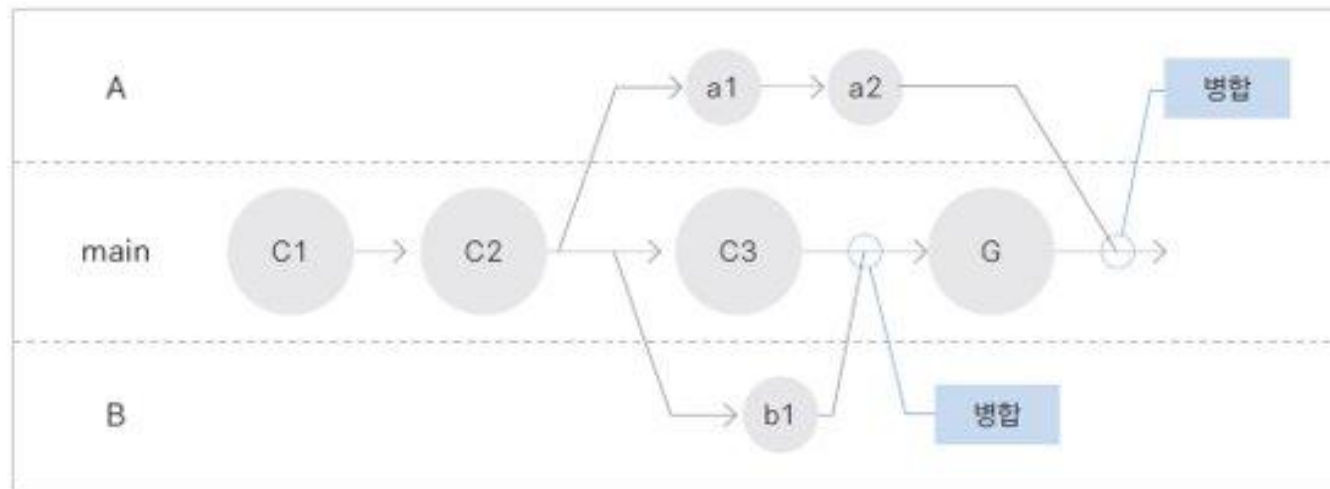


↓ **푸시 (push, fetch)**



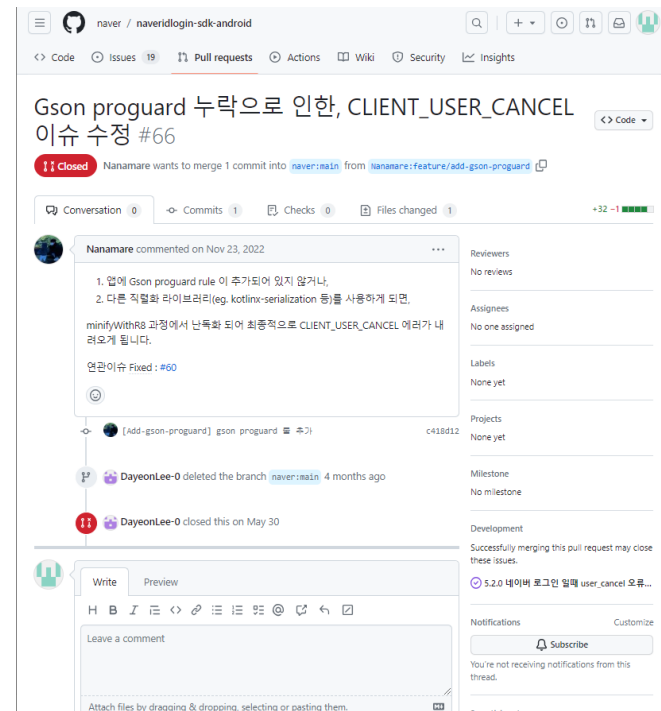
# 용어 정리 (2/5)

- 분기(branch) = **협업의 핵심!**
  - 프로젝트를 분리해 여러 작업을 동시에 수행, 관리하기 위한 메커니즘
  - **main** : 기본 branch (예전엔 master)
  - **A/B** : 서브 branch, 개발자 별 (A, B, ...), 작업 목표 별 (기능 A 구현, 버그 B 해결, ...)
  - **병합 (merge)**: 서브 branch의 수정 사항을 main branch로 합치는 작업



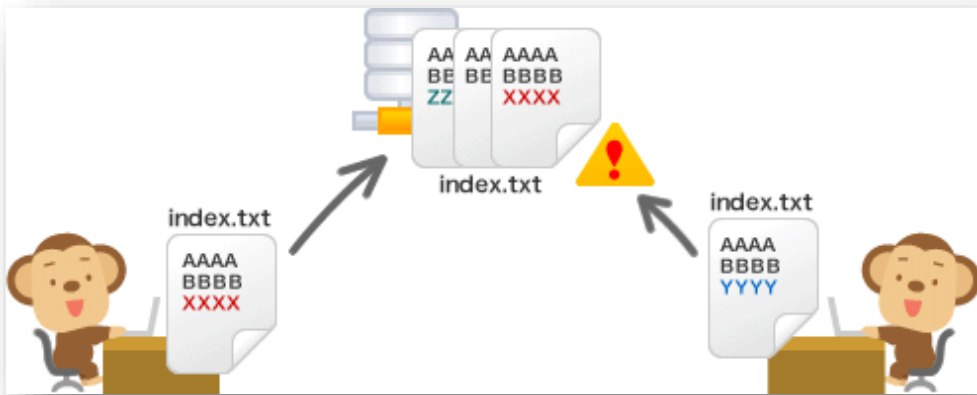
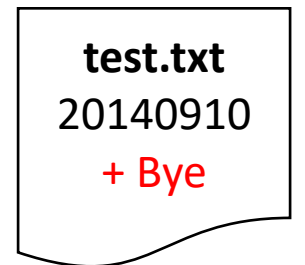
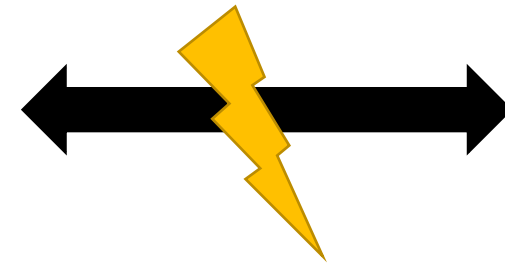
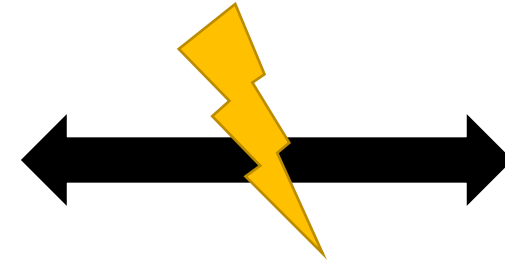
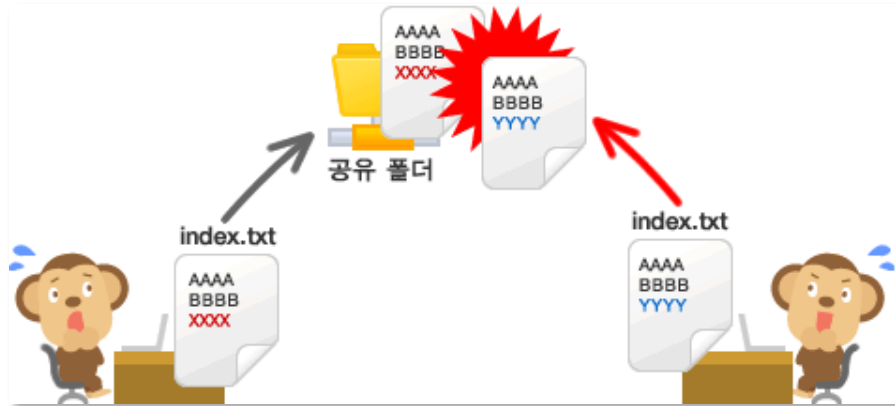
브랜치 병합 과정

- Pull request: 코드 변경 사항을 리뷰하고 병합(merge)하기 위한 메커니즘

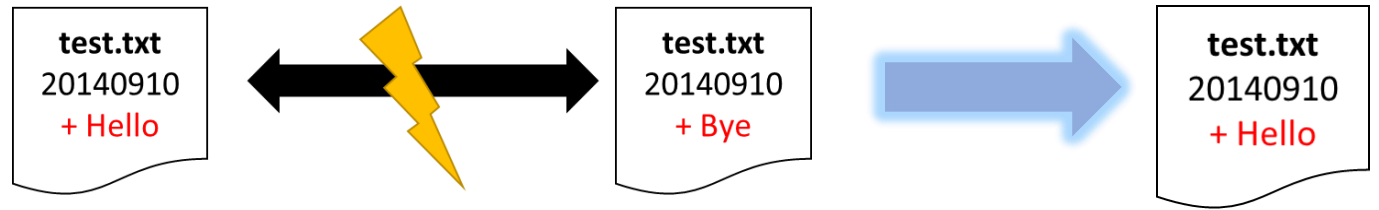


# 용어 정리 (4/5)

- Conflict: 동일한 내용을 각자 다르게 변경한 이력이 있을 때, 변경 이력들끼리 발생하는 충돌

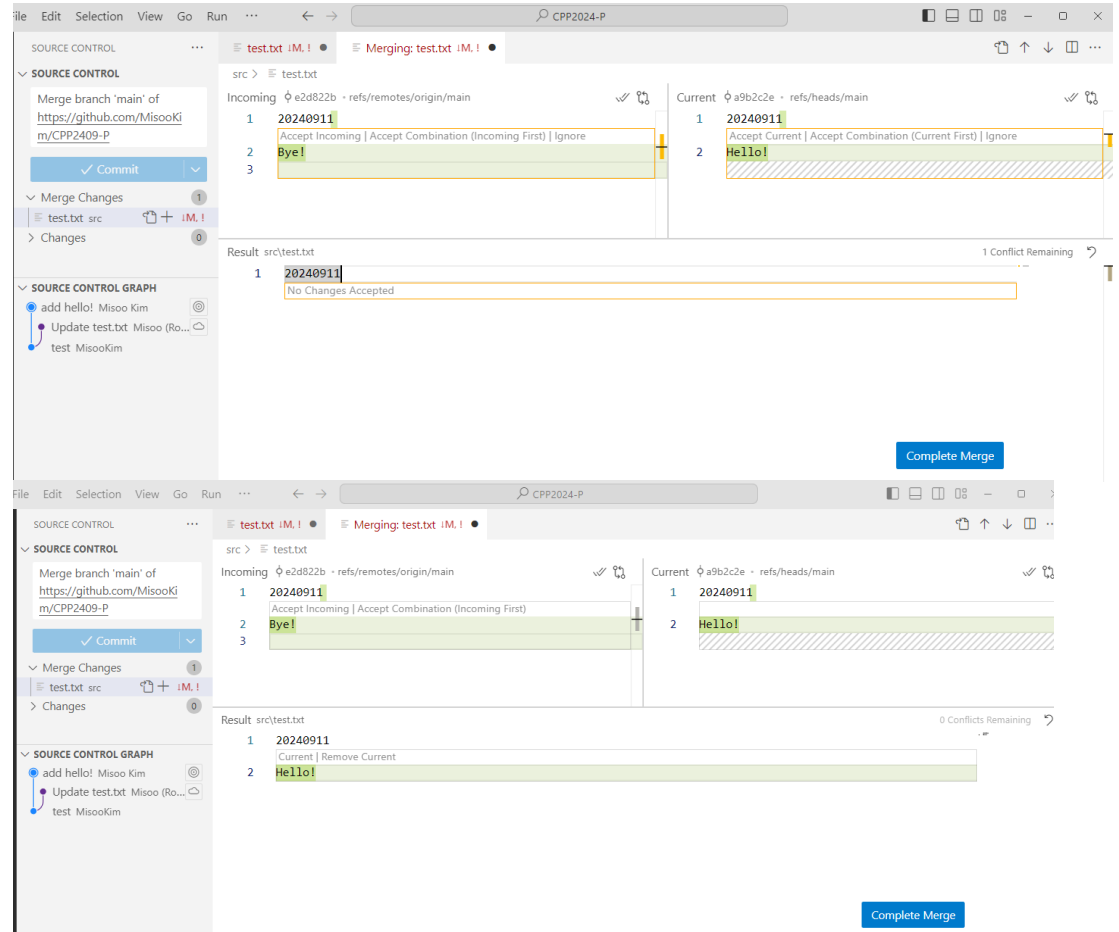
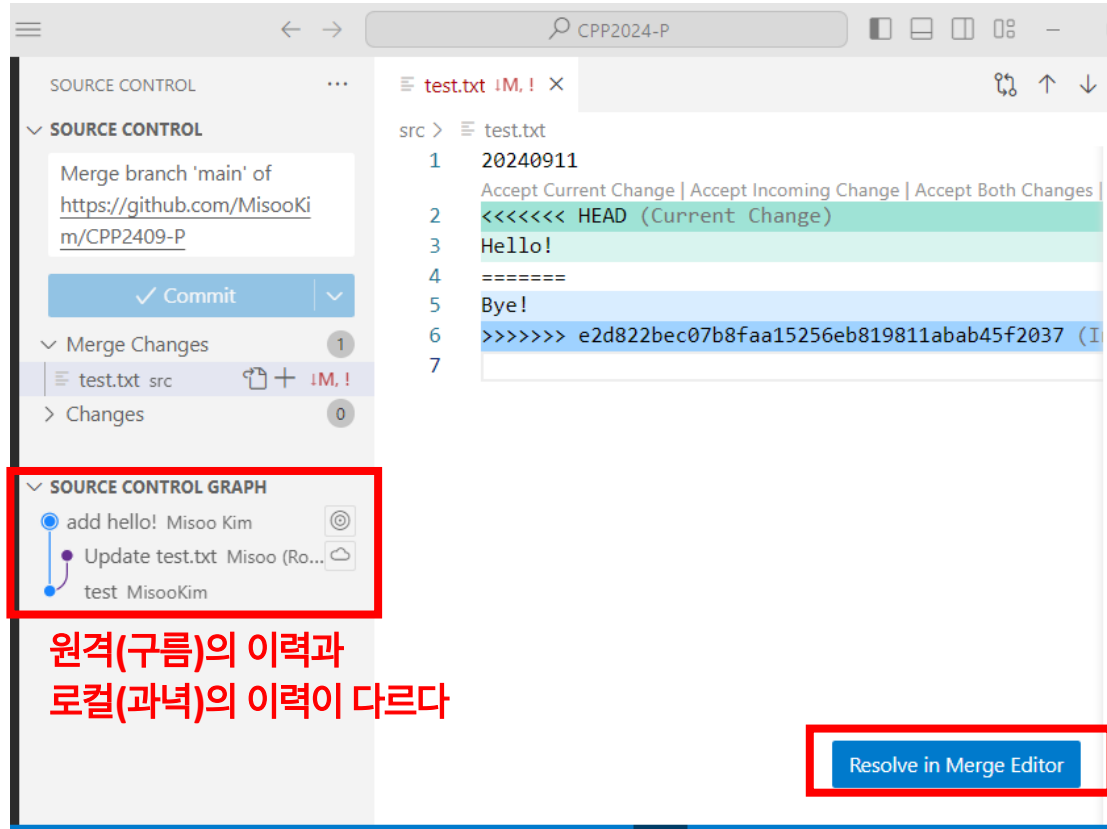


# 용어 정리 (5/5)



- Conflict가 발생하면, 해당 파일에 부딪힌 코드들이 병합되며, 개발자의 수정을 요구함

▶ 개발자는 두 변경 중 하나를 택하거나 통합하는 등의 수정한 후 커밋해야 함



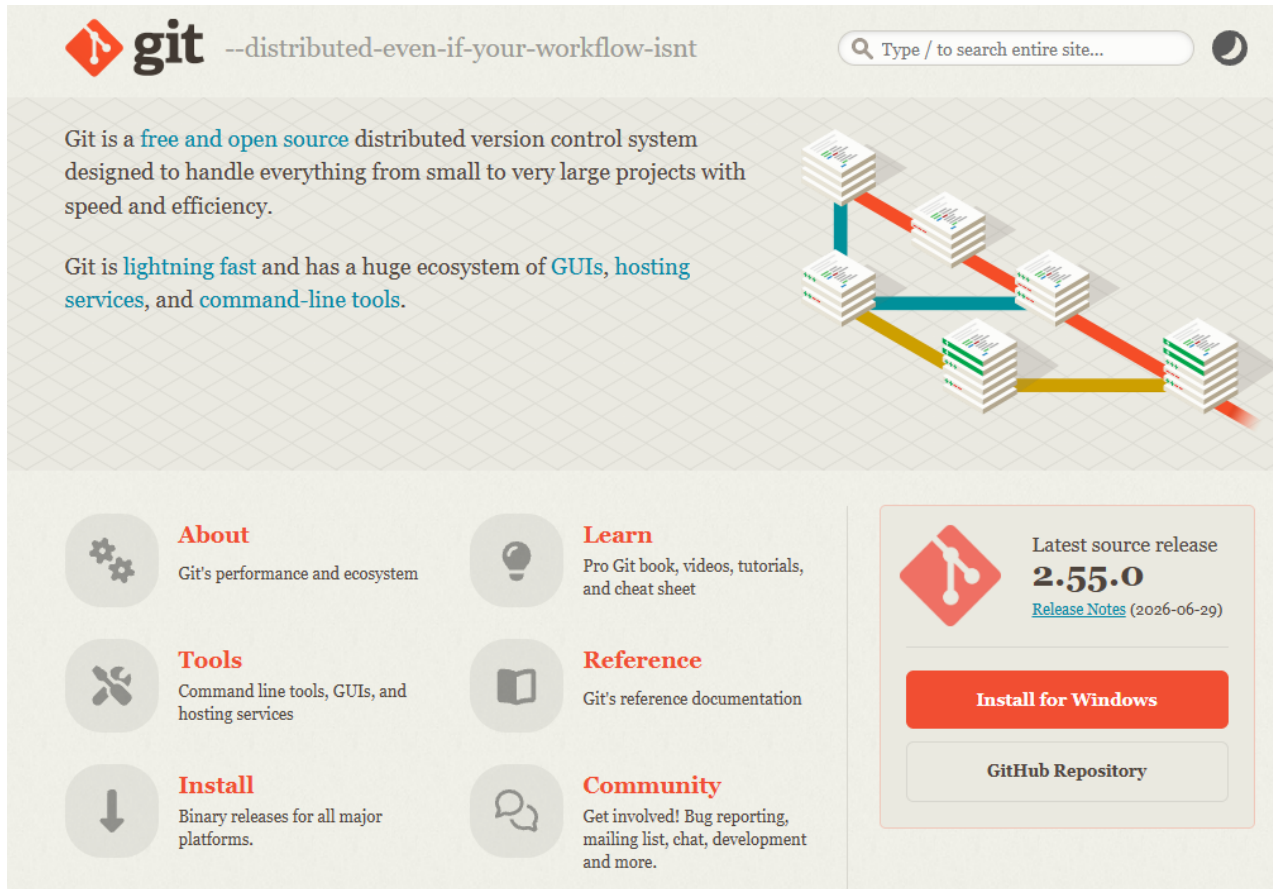
# git과 github 시작해보기

1. git 설치
2. github 가입
3. 내 컴퓨터에 지역저장소 만들기
4. 내 컴퓨터에 지역저장소 만들고 올리기

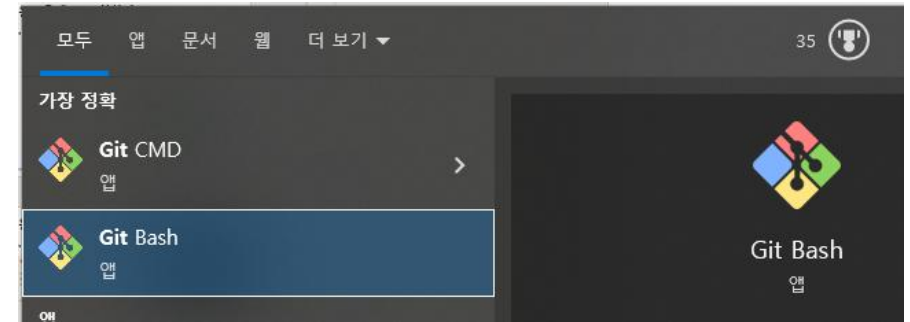
# git과 github 시작해보기

## 1. Git 설치 후 확인

- <https://git-scm.com/>



The image shows the Git website homepage. At the top, the Git logo is followed by the tagline "--distributed-even-if-your-workflow-isnt". Below this, a search bar and a moon icon are visible. The main content area features a diagram of a distributed version control system with multiple nodes connected by lines. Text on the left describes Git as a "free and open source distributed version control system" and "lightning fast". Below this, there are links for "GUIs, hosting services, and command-line tools". At the bottom, there are several sections: "About" (Git's performance and ecosystem), "Tools" (Command line tools, GUIs, and hosting services), "Install" (Binary releases for all major platforms), "Learn" (Pro Git book, videos, tutorials, and cheat sheet), "Reference" (Git's reference documentation), and "Community" (Get involved! Bug reporting, mailing list, chat, development and more.). On the right side, there is a section for the "Latest source release 2.55.0" with a link to "Release Notes (2026-06-29)" and buttons for "Install for Windows" and "GitHub Repository".



```
MINGW64:/c/Users/tnswh

tnswh@DESKTOP-MSH09QO MINGW64 ~
$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
          [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
          [--config-env=<name>=<envvar>] <command> [<args>]

These are common Git commands used in various situations:


start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

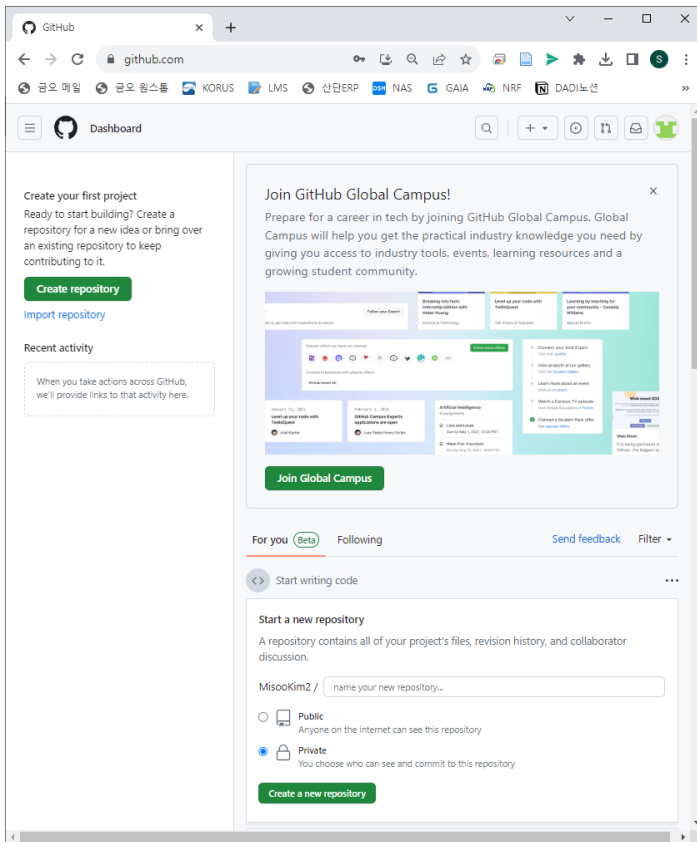
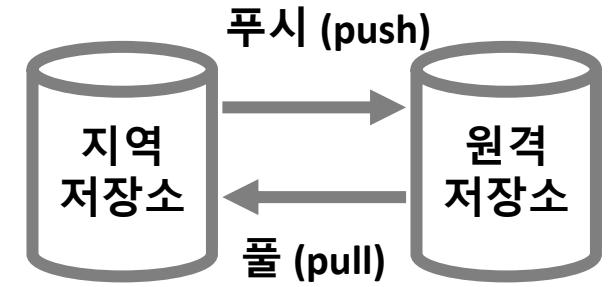

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index


examine the history and state (see also: git help revisions)
  bisect    Use binary search to find the commit that introduced a bug
  diff      Show changes between commits, commit and working tree, etc
  grep      Print lines matching a pattern
  log       Show commit logs
  show      Show various types of objects
  status    Show the working tree status
```

# git과 github 시작해보기

## 2. GitHub 가입 후 원격 저장소 만들기

- 저장소의 종류: 지역(local) vs 원격(remote)



### Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#). Required fields are marked with an asterisk (\*).

#### 1 General

Owner \* / Repository name \*   
✔ 26-2\_cpp is available.

Great repository names are short and memorable. How about [fuzzy-octo-palm-tree?](#)

#### Description

0 / 350 characters

#### 2 Configuration

##### Choose visibility \*

Choose who can see and commit to this repository

Public

##### Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

##### Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

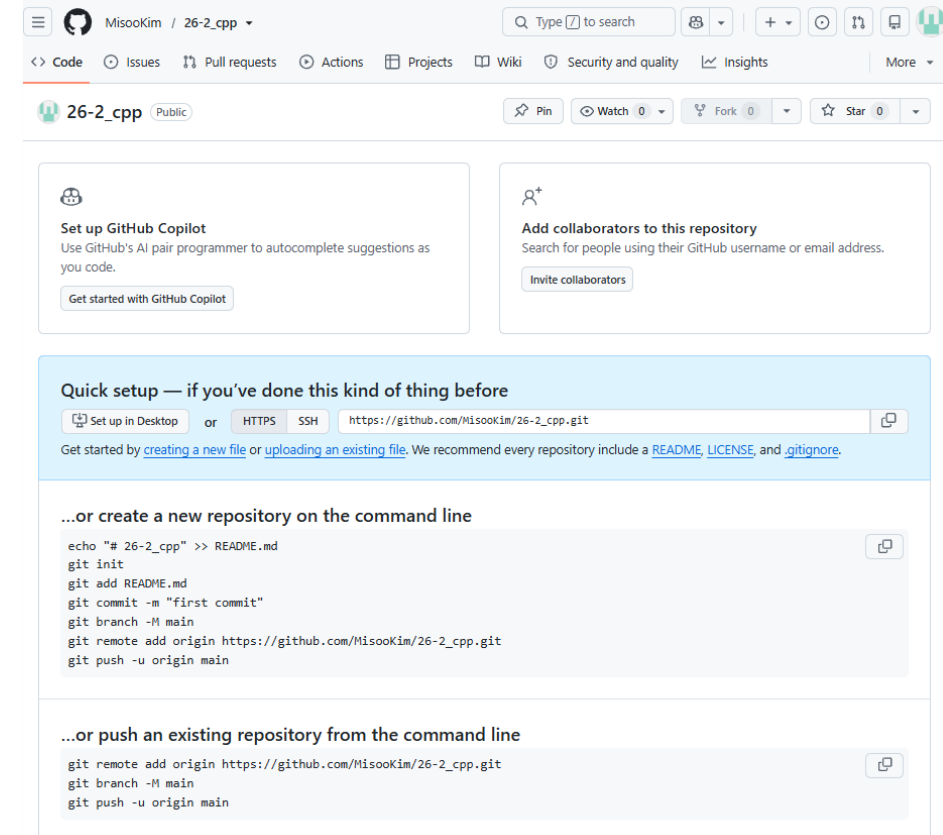
No .gitignore

##### Add license

Licenses explain how others can use your code. [About licenses](#)

No license

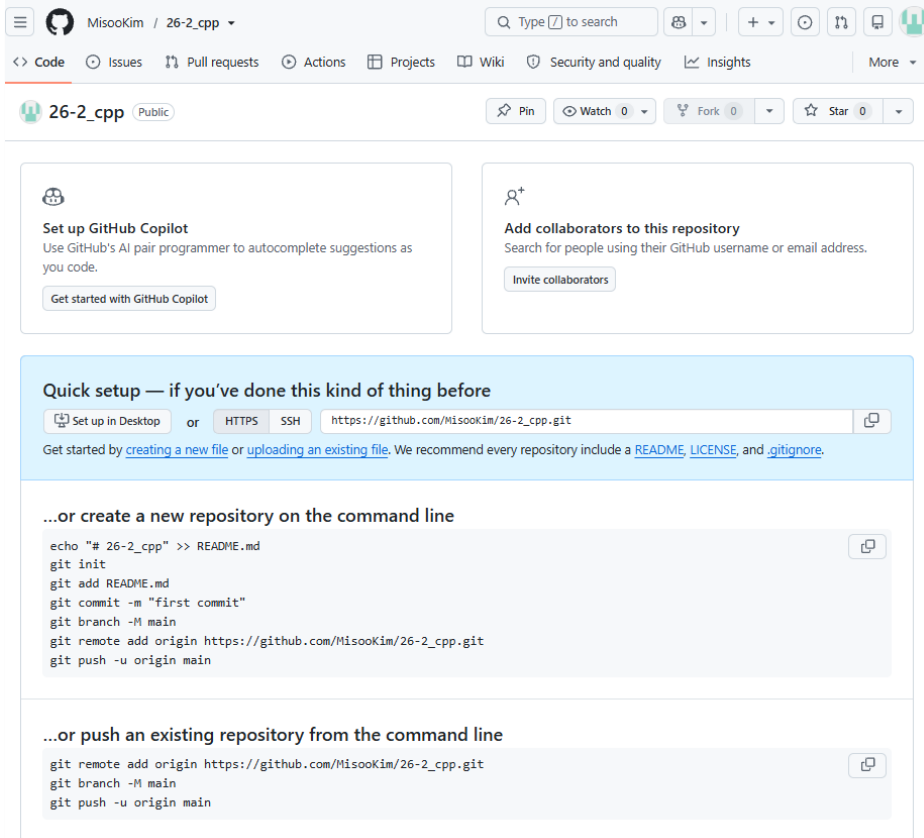
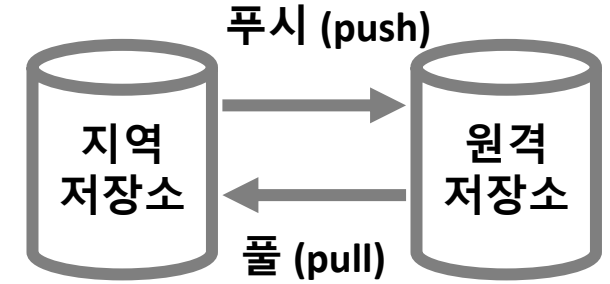
Create repository



# git과 github 시작해보기

## 3. 내 컴퓨터에 지역 저장소 만들어보기

- git init부터 복사 후 붙여넣기



```
test.cpp U X
test.cpp > ...
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Hello, World!" << endl;
6     return 0;
7 }
8

Problems Output Debug Console Terminal Ports

C:\cpp_practice\2-1>git init
Initialized empty Git repository in C:/cpp_practice/2-1/.git/

C:\cpp_practice\2-1>git add README.md
fatal: pathspec 'README.md' did not match any files

C:\cpp_practice\2-1>git commit -m "first commit"
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .vscode/
        test.cpp
        test.exe
        test2.exe

nothing added to commit but untracked files present (use "git add" to track)

C:\cpp_practice\2-1>git branch -M main

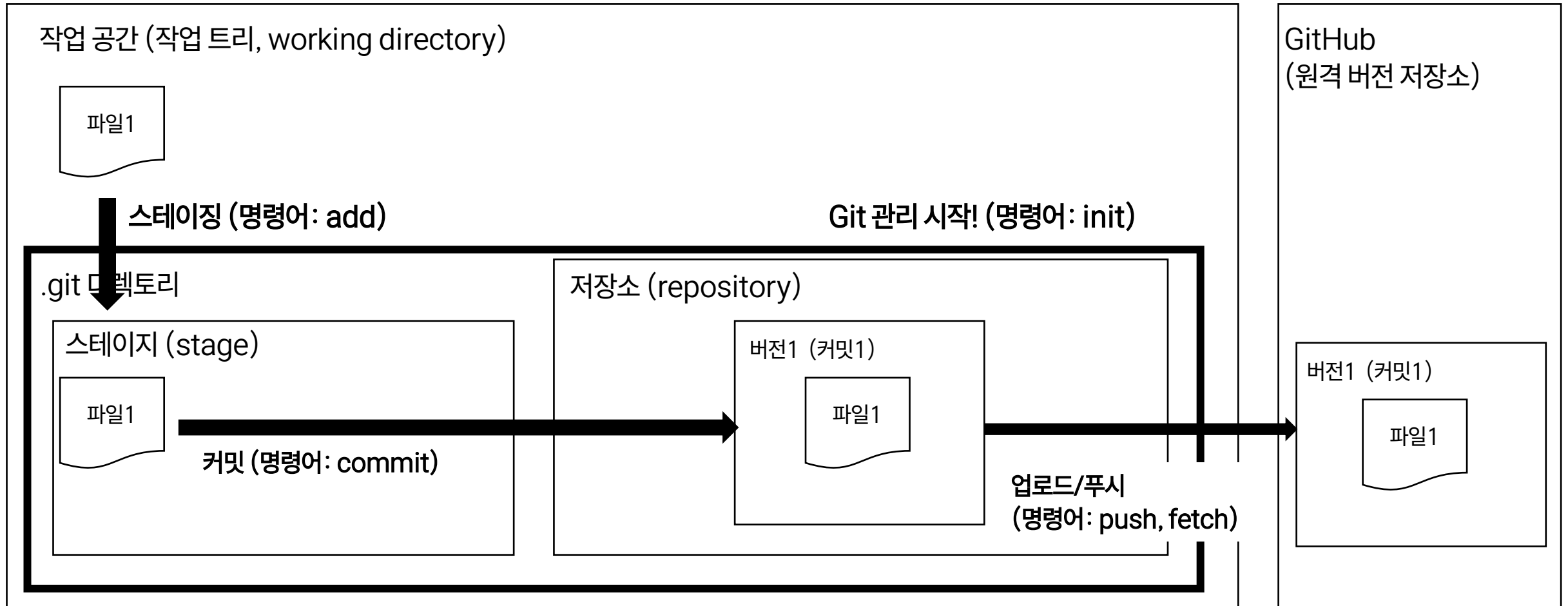
C:\cpp_practice\2-1>git remote add origin https://github.com/MisookKim/26-2_cpp.git

C:\cpp_practice\2-1>git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/MisookKim/26-2_cpp.git'
```

**Why?**  
**> add 로 추가하려는 파일이 없다!**

# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어

GitHub 내에서  
저장소 생성



# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어

tracked 상태  
git이 버전으로 관리할 파일  
(, working directory)

파일1

파일2

untracked 상태  
git이 버전으로 관리하지 않는 파일

↓ 스테이징 (명령어: git add 파일1)

.git 디렉토리

스테이지 (stage)

파일1

저장소 (repository)

GitHub  
(원격 버전 저장소)

# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어

tracked 상태  
git이 버전으로 관리할 파일  
, working directory)

파일1

파일2

tracked 상태  
git이 버전으로 관리할 파일

스테이징 (명령어: git add .)

.git 디렉토리

스테이지 (stage)

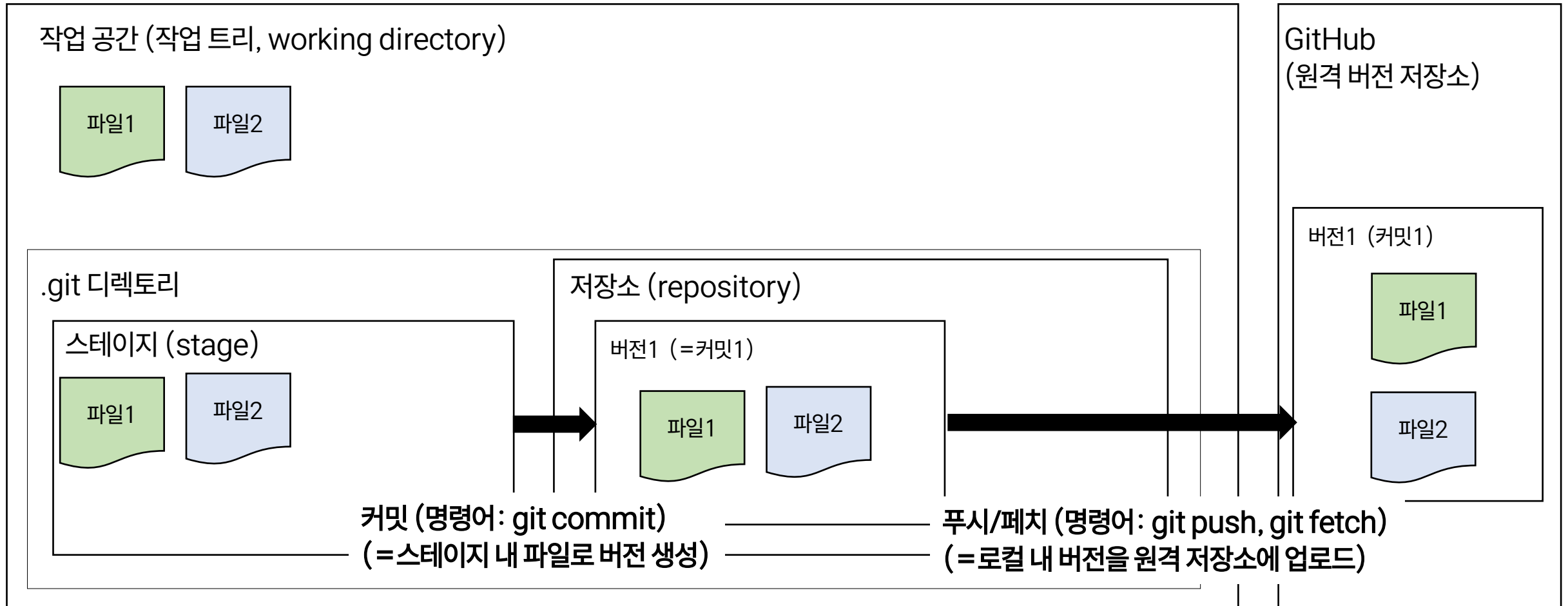
파일1

파일2

저장소 (repository)

GitHub  
(원격 버전 저장소)

# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어

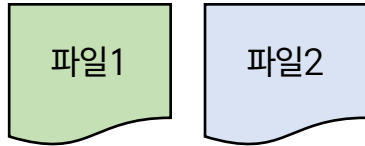


# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어

동료  
개발자

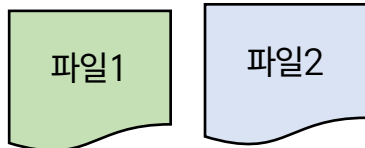


작업 공간 (작업 트리, working directory)



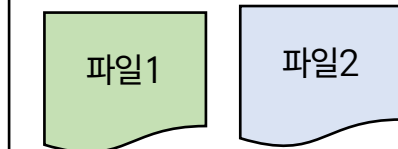
.git 디렉토리

스테이지 (stage)



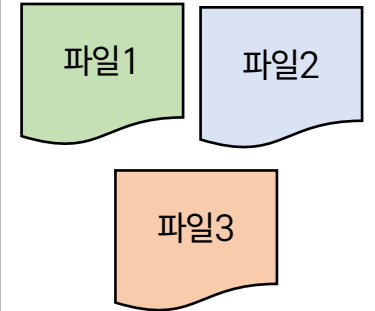
저장소 (repository)

버전1 (=커밋1)

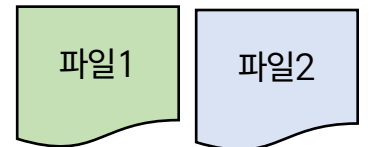


GitHub  
(원격 버전 저장소)

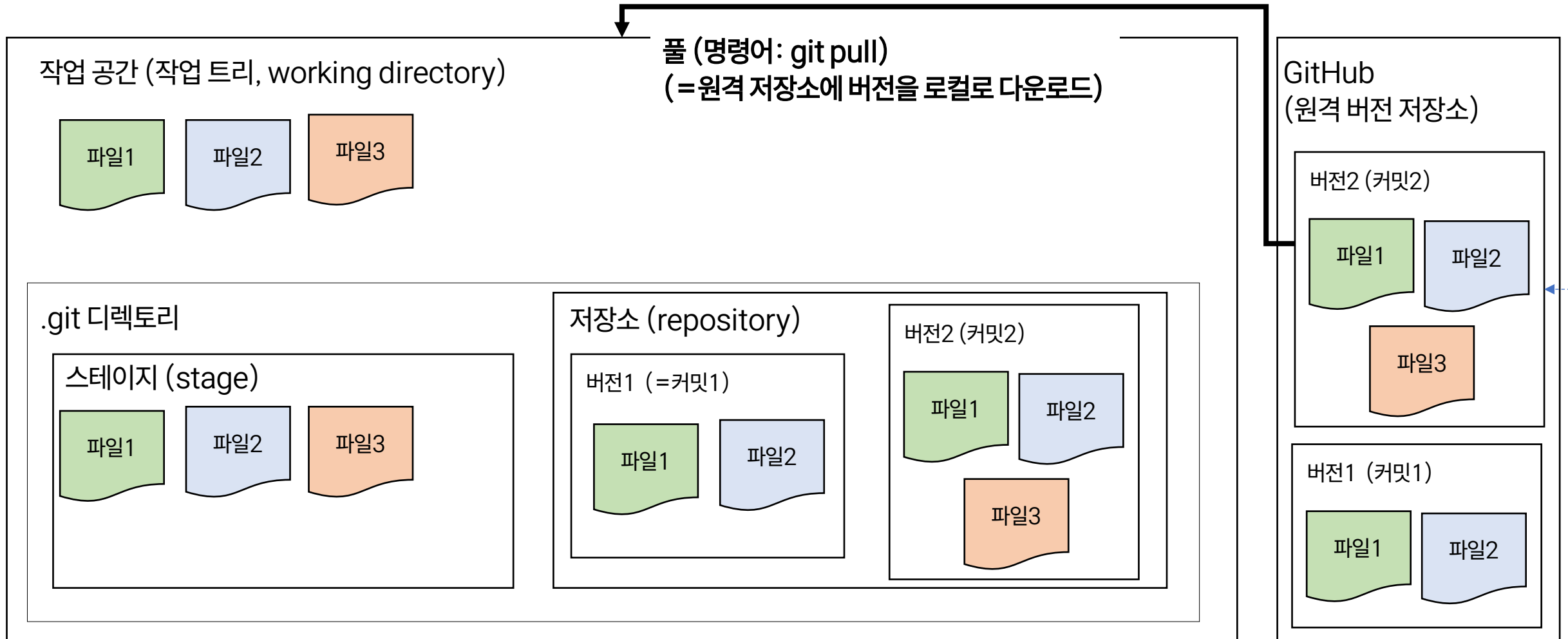
버전2 (커밋2)



버전1 (커밋1)



# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어



# git과 github 시작해보기 - Git의 버전 관리 원리와 명령어

작업 공간 (작업 트리, working directory)

**파일 생성/수정/삭제 영역**

.git 디렉토리

스테이지 (stage)

**버전 생성 전  
파일을 임시로 보관**

저장소 (repository)

버전1 (=커밋1)

버전2 (커밋2)

**버전(=커밋)을  
저장해두는 곳**

GitHub  
(원격 버전 저장소)

버전1 (커밋1)

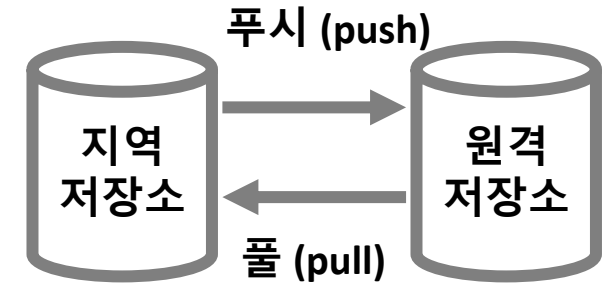
**온라인  
버전  
저장소**

버전2

# git과 github 시작해보기

## 4. 내 컴퓨터에 지역 저장소 만들고 올리기

- 저장소 생성 및 초기화를 위한 기본 명령어



```
C:\cpp_practice\2-1>git add .  
warning: in the working copy of '.vscode/tasks.json', LF will be replaced by CRLF the next time Git touches it
```

```
C:\cpp_practice\2-1>git commit -m "first commit"  
[main (root-commit) cf25377] first commit  
4 files changed, 35 insertions(+)  
create mode 100644 .vscode/tasks.json  
create mode 100644 test.cpp  
create mode 100644 test.exe  
create mode 100644 test2.exe
```

```
C:\cpp_practice\2-1>git branch -M main
```

```
C:\cpp_practice\2-1>git remote add origin https://github.com/MisooKim/26-2_cpp.git  
error: remote origin already exists.
```

```
C:\cpp_practice\2-1>git push -u origin main  
remote: Permission to MisooKim/26-2_cpp.git denied to MisooKim2.  
fatal: unable to access 'https://github.com/MisooKim/26-2_cpp.git/': The requested URL returned error: 403
```

```
git init  
git add .  
git commit -m "first commit"  
git branch -M main  
git remote add origin https://github.com/MisooKim/26-2_cpp.git  
git push -u origin main
```

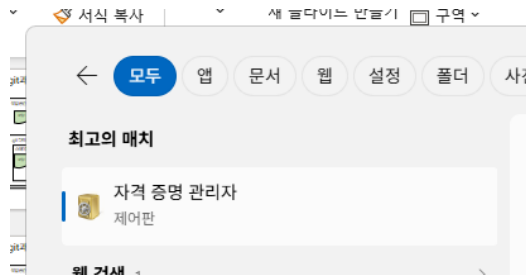
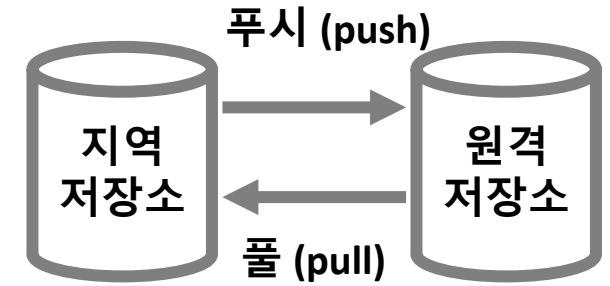
**Why?**

**> 올릴 권한이 없다!**

# git과 github 시작해보기

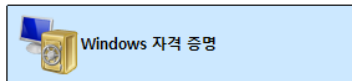
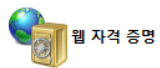
## 4. 내 컴퓨터에 지역 저장소 만들고 올리기

- 공용 PC에 기존 github 자격 증명 제거 → 다시 push → github 로그인



### 자격 증명 관리

웹 사이트, 연결된 응용 프로그램 및 네트워크에 대해 저장된 로그인 정보를 보고 삭제합니다.



자격 증명 백업(B) 자격 증명 복원(R)

Windows 자격 증명

Windows 자격 증명 추가

Windows 자격 증명이 없습니다.

인증서 기반 자격 증명

인증서 기반 자격 증명 추가

인증서가 없습니다.

일반 자격 증명

일반 자격 증명 추가

git:https://github.com

수정한 날짜: 2026-05-21

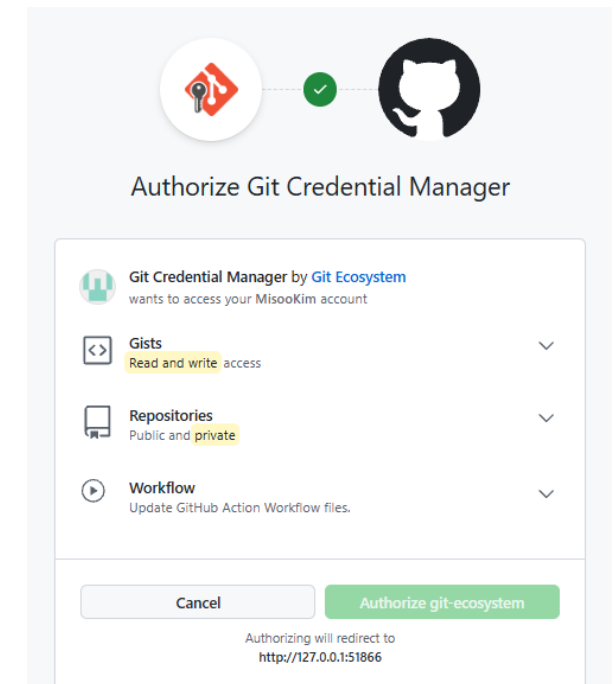
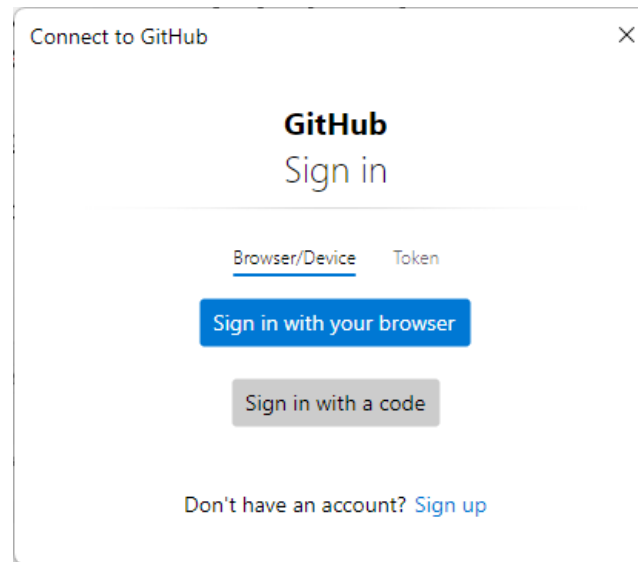
인터넷 또는 네트워크 주소: git:https://github.com

사용자 이름: MisookKim2

암호: .....

지속성: 로컬 컴퓨터

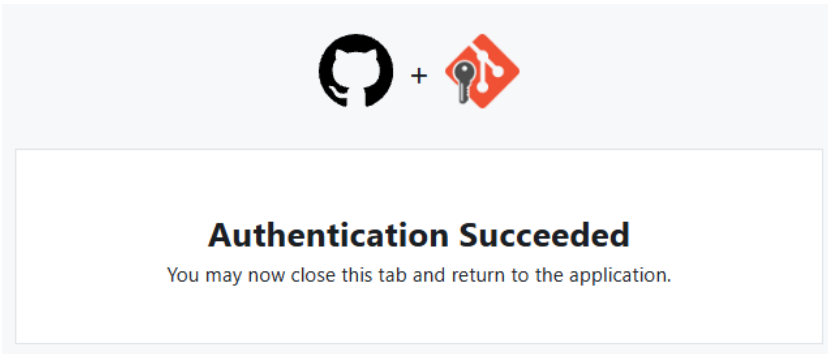
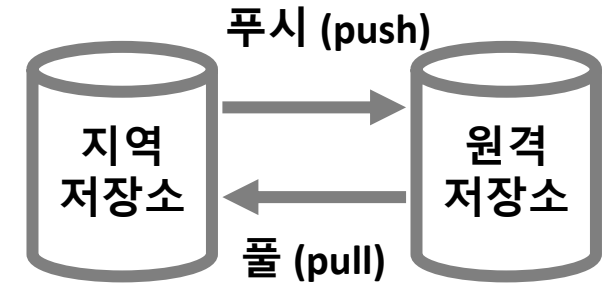
편집 제거



# git과 github 시작해보기

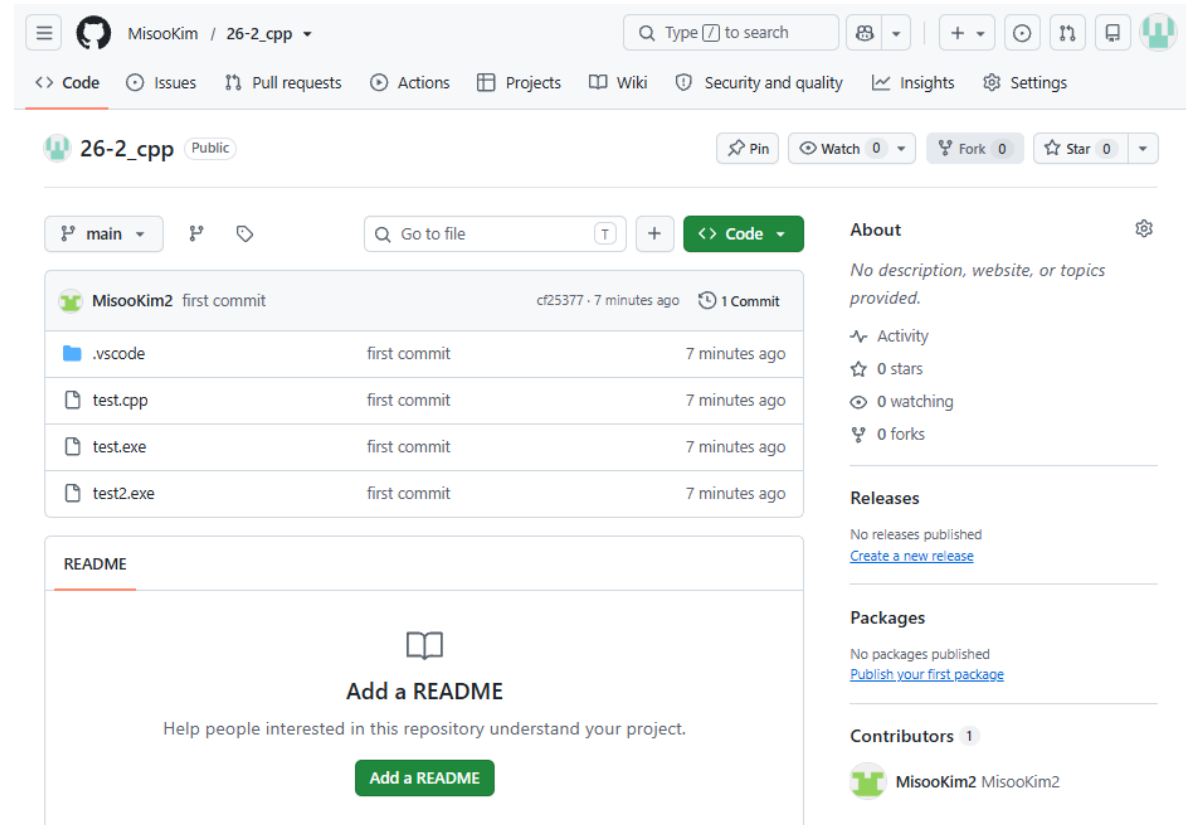
## 4. 내 컴퓨터에 지역 저장소 만들고 올리기

- 공용 PC에 기존 github 자격 증명 제거 → 다시 push → github 로그인



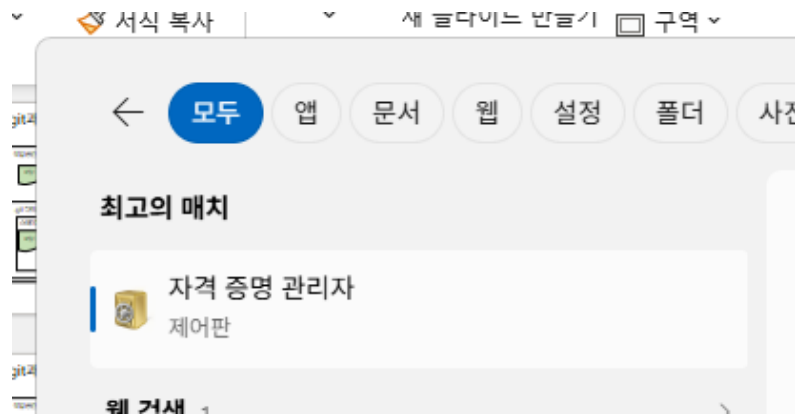
```
data compression using up to 25 mB/s
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 41.30 KiB | 5.16 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/MisooKim/26-2_cpp.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

```
C:\cpp_practice\2-1>
```



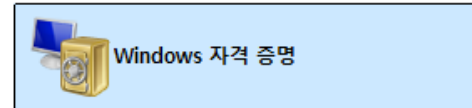
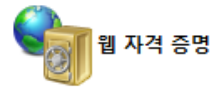
# git과 github 시작해보기

- 공용 PC 사용 시 주의 사항!!!
- 꼭 여러분들의 자격증명을 제거하고 가세요~~~!!!



## 자격 증명 관리

웹 사이트, 연결된 응용 프로그램 및 네트워크에 대해 저장된 로그인 정보를 보고 삭제합니다.



[자격 증명 백업\(B\)](#) [자격 증명 복원\(R\)](#)

Windows 자격 증명

[Windows 자격 증명 추가](#)

Windows 자격 증명이 없습니다.

인증서 기반 자격 증명

[인증서 기반 자격 증명 추가](#)

인증서가 없습니다.

일반 자격 증명

[일반 자격 증명 추가](#)

git:https://github.com

수정된 날짜: 2026-05-21 ^

인터넷 또는 네트워크 주소: git:https://github.com

사용자 이름: MisooKim2

암호: .....

지속성: 로컬 컴퓨터

[편집](#) [제거](#)

감사합니다.  
Q & A